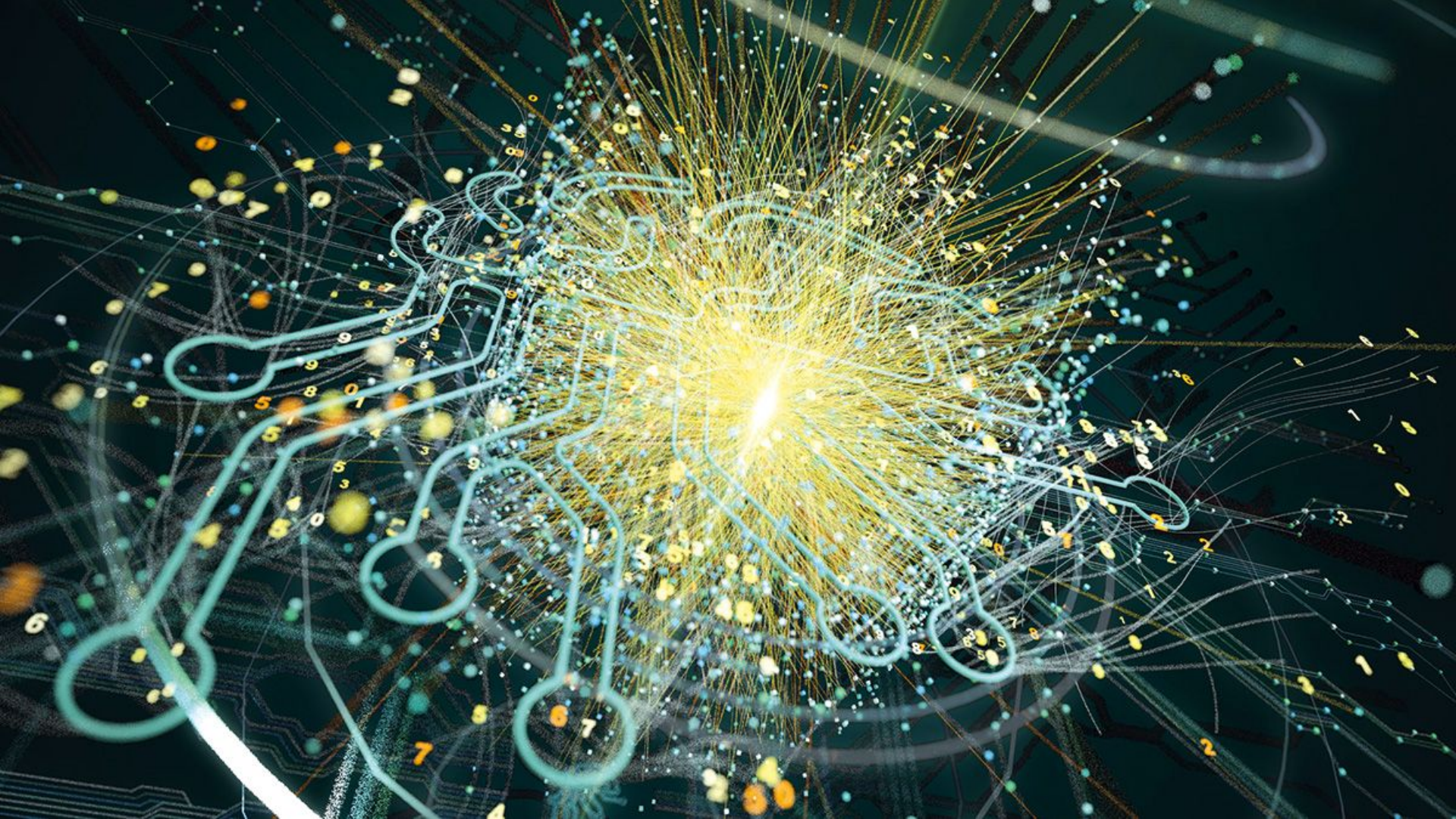
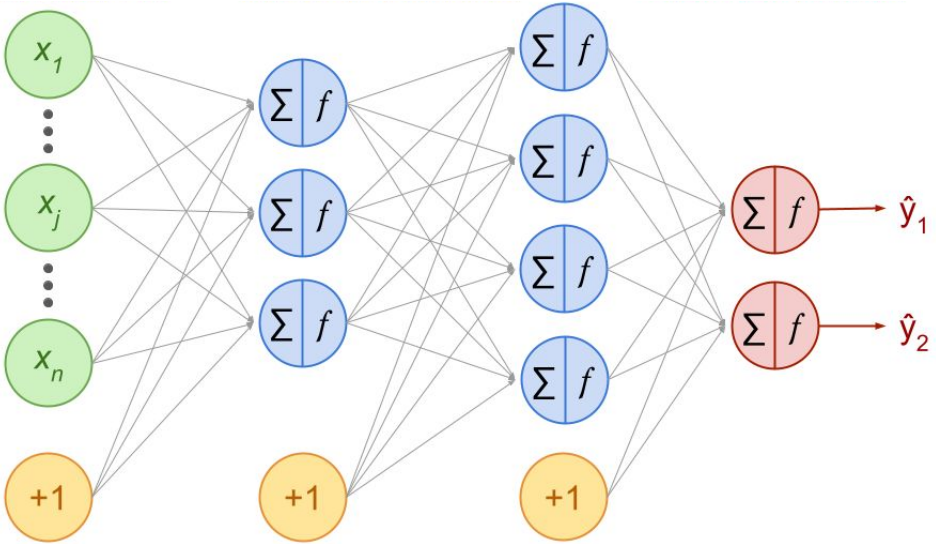


The Math Behind Neural Networks



Pelumi, Catherine & Claire David

AIMS African Institute for Mathematical Sciences SOUTH AFRICA

Introduction

What is machine learning?

ARTIFICIAL INTELLIGENCE

Science & engineering of making intelligent machines

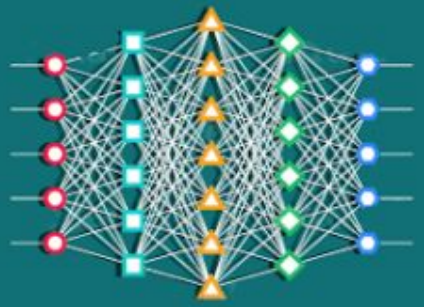
Example: chatbots

MACHINE LEARNING

"Field of study that gives computers the ability to learn without being explicitly programmed"

Arthur Samuel

DEEP LEARNING



Learning based on Deep Neural Networks

Machine Learning

Definition

A computer program is said to learn from **experience E** with respect to some **task T** and some **performance measure P**,
if its performance on T, as measured by P, improves with experience E.

Tom Mitchell, computer scientist, 1998

Important!

- Assessment (win / lose) should be done on **new data**
- What counts in the experience is **novelty**

Example: checker

Task: playing a checker game

Experience: repeated action of the program to play against itself for thousands of different games

Performance: probability of winning the next game of checker.

Example: spam filter

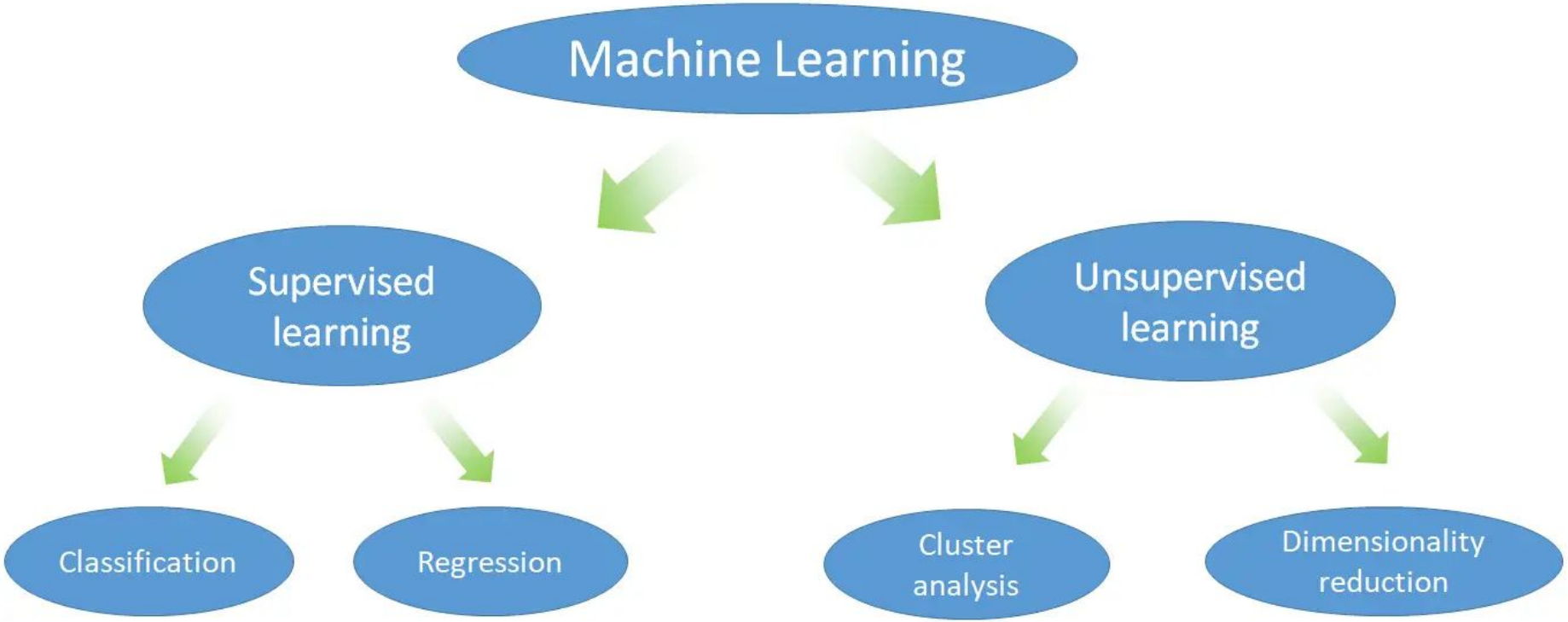
Task: marking an email either spam or not-spam

Experience: repeating the task with a large collection of emails of known type

Performance: accuracy = percentage of correct decisions / all decisions

Machine Learning is a tool

Types of Machine Learning

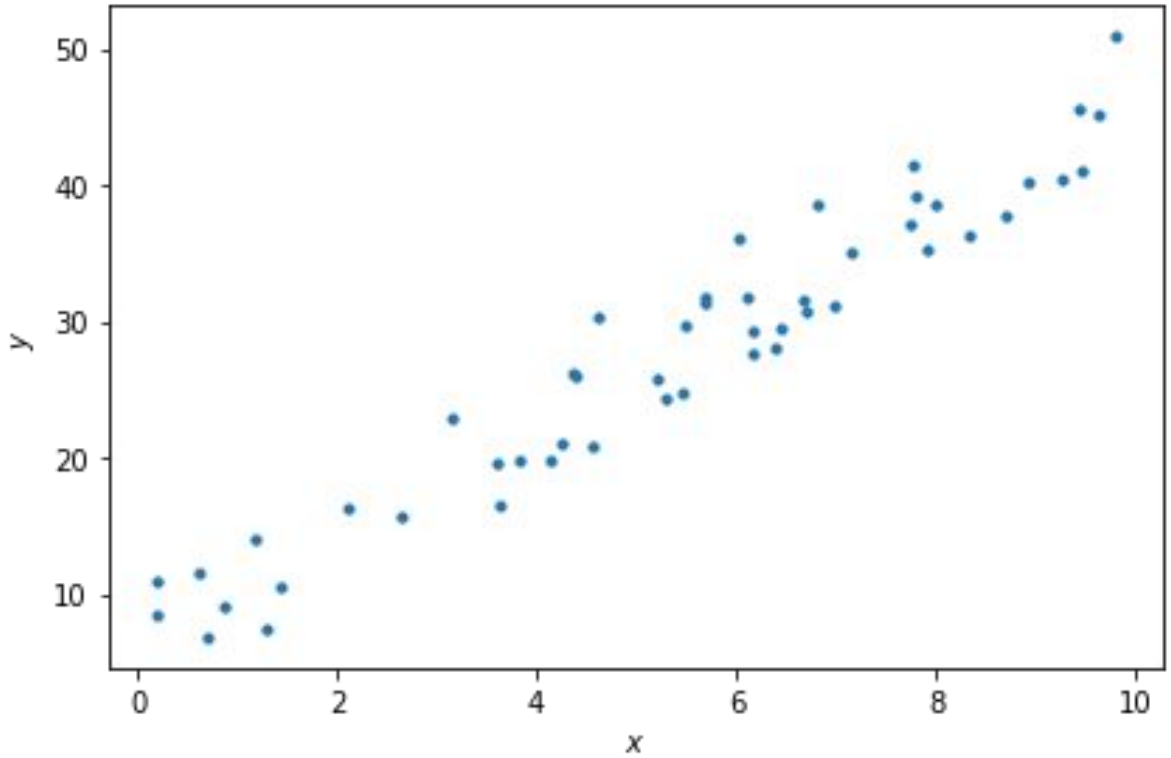


Linear Regression

Let’s speak ML

Gradient Descent in 1 dimension

"Fitting a straight line"



Gradient Descent

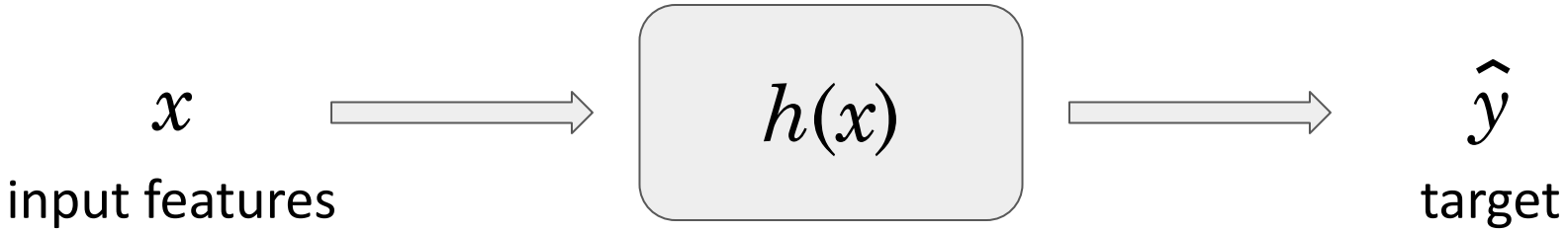
Definition

Iterative optimization algorithm to find the minimum of a function.

The hypothesis function h

Definition

Mapping function used to predict an output from an input.



Cost Function in Linear Regression

Definition

The **cost function** in linear regression returns a global error between the predicted values from a mapping function h (predictions) and all the target values (observations) of the training data set.

In linear regression, a common cost function is the Mean Squared Error (MSE):

$$\text{Cost} = \frac{1}{m} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

prediction target

Rephrasing the goal

Hypothesis function in 1 dimension: $\hat{y} = wx + b$

Hypothesis function with n features: $\hat{y} = \sum_{j=1}^{n} w_j x_j + b$

"Fit the data"

Find parameters

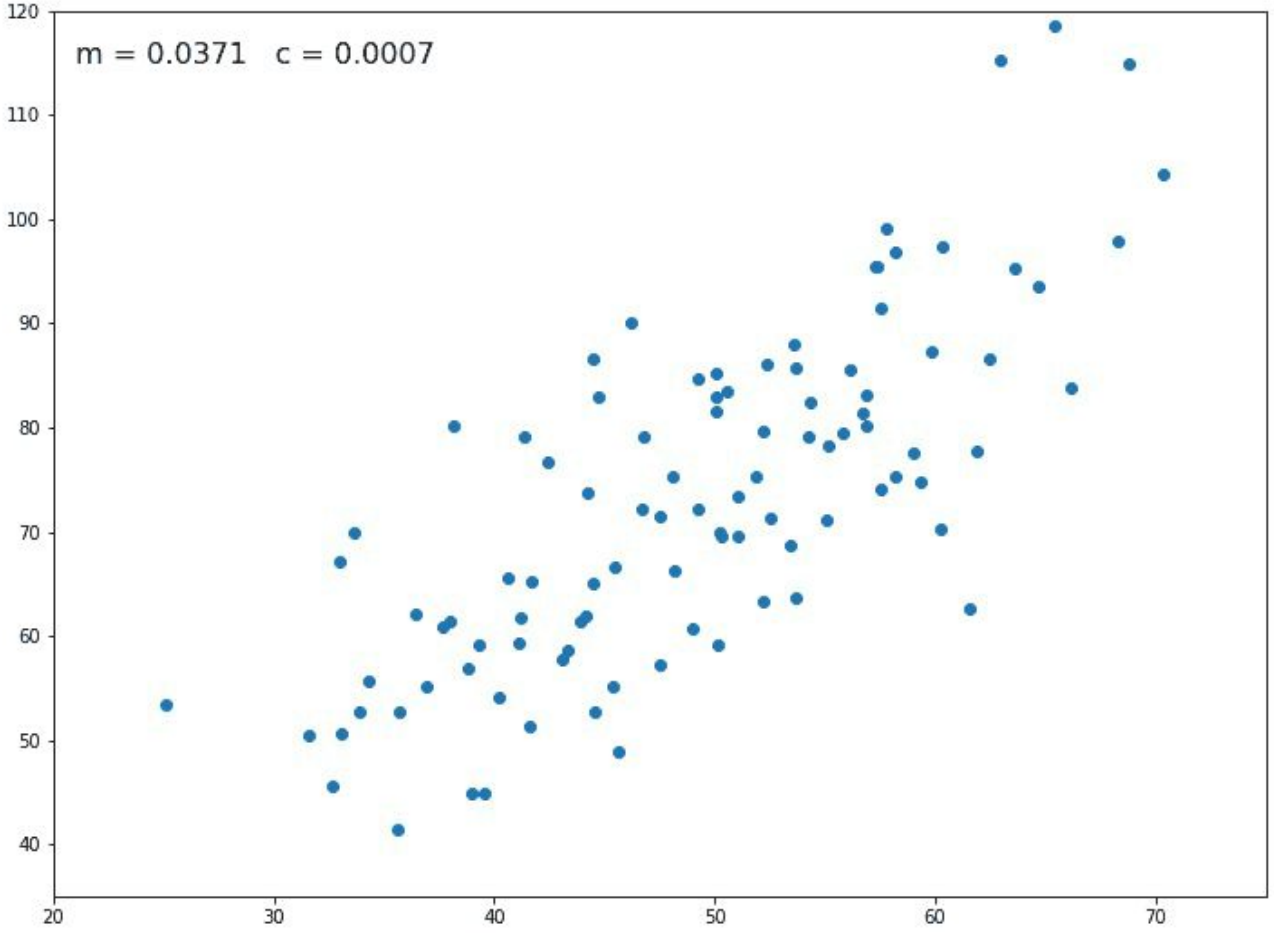
w and b

minimizing the cost

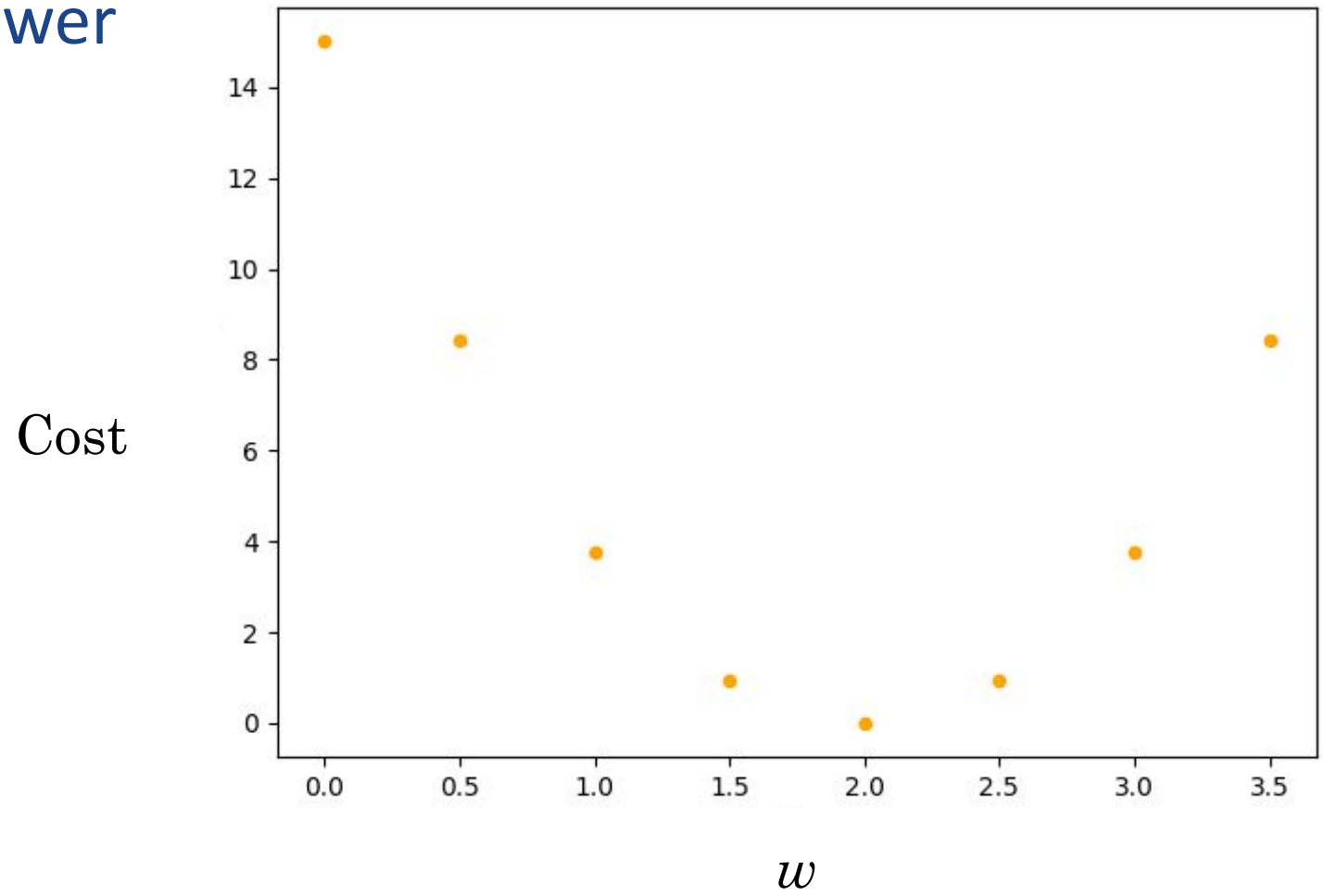
Visualizing the cost

If b = 0, how does the cost vary with respect to w?

Linear Regression: the movie



Answer



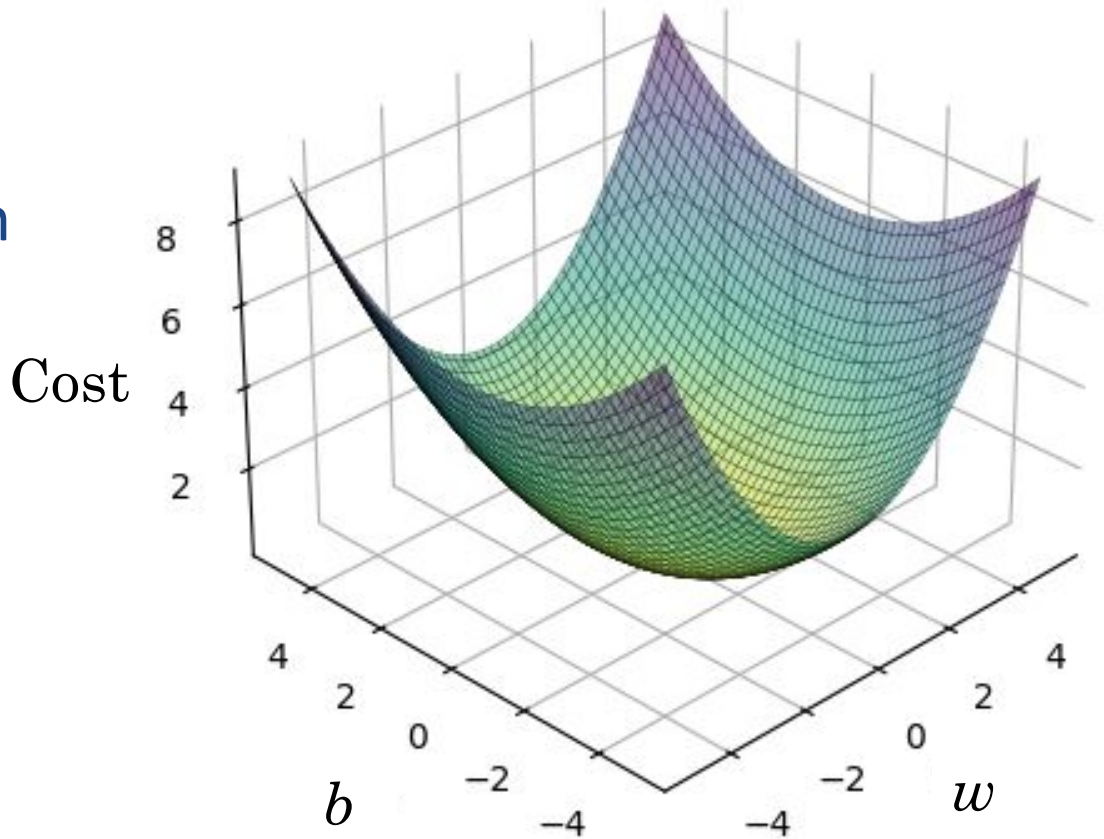
Cost in linear regression

1 global minimum (convex function)

Our goal:

go toward the minimum

of the cost



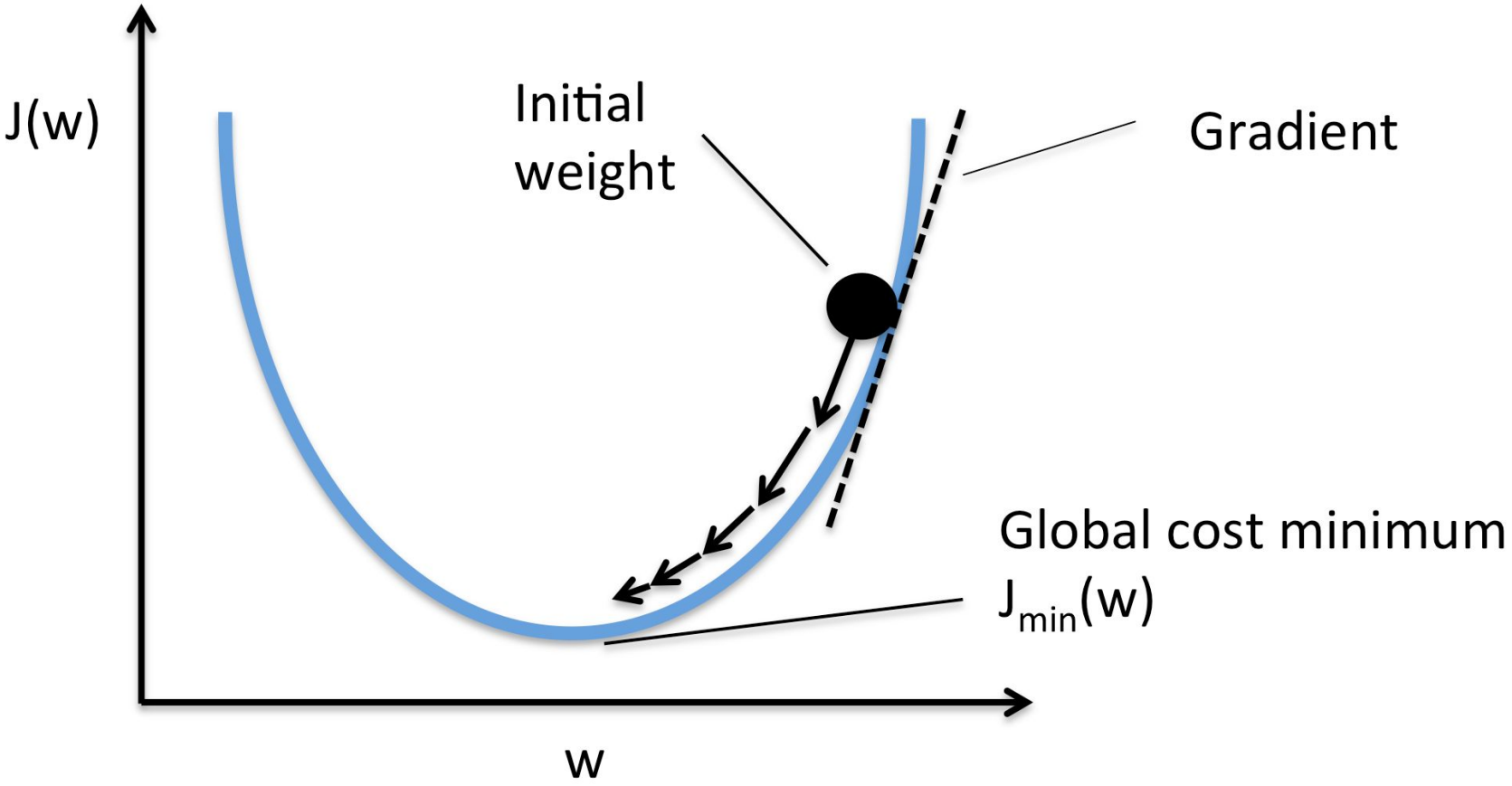
But each point is computed

with the entire dataset!

Gradient Descent

Going down a valley blanketed by thick fog

Gradient Descent Algorithm



Gradient Descent Algorithm

Init: Pick a random (W , b)

Repeat:

Compute partial derivatives $\frac{\partial \text{Cost}}{\partial W}$ $\frac{\partial \text{Cost}}{\partial b}$

Update the parameters:

$$W' = W - \alpha \frac{\partial \text{Cost}}{\partial W} \qquad b' = b - \alpha \frac{\partial \text{Cost}}{\partial b}$$

Reassign: $W = W' \quad b = b'$

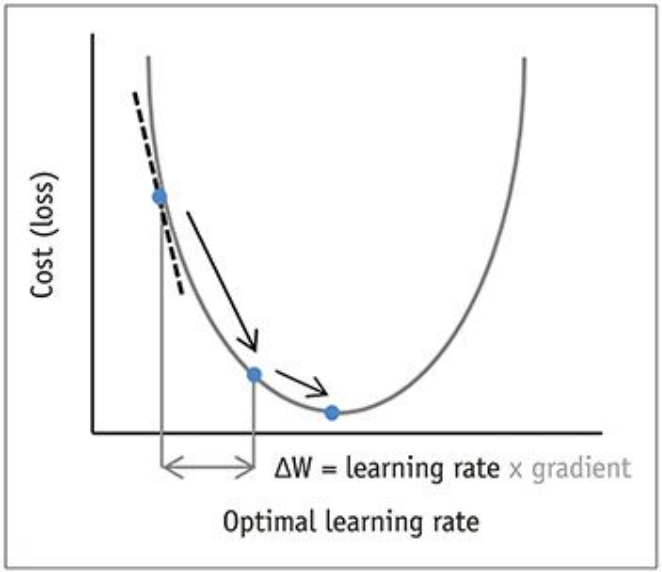
Hyperparameter α = learning rate

Until N iterations or both derivatives $\rightarrow 0$

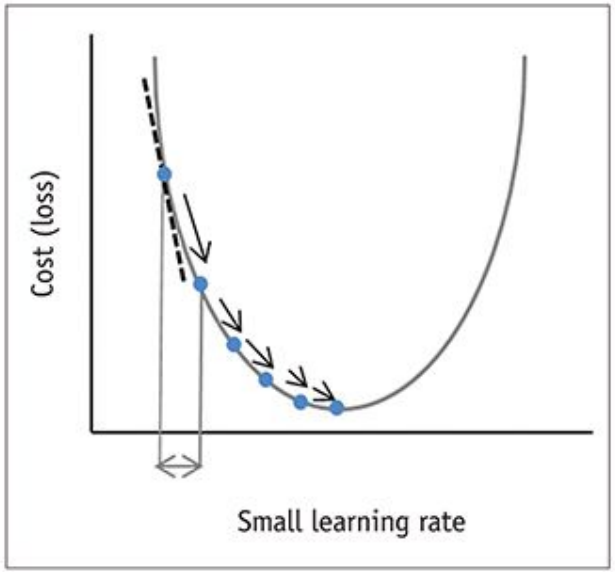
Return: Optimized values of (W^{opt}, b^{opt})

Choosing the Learning Rate

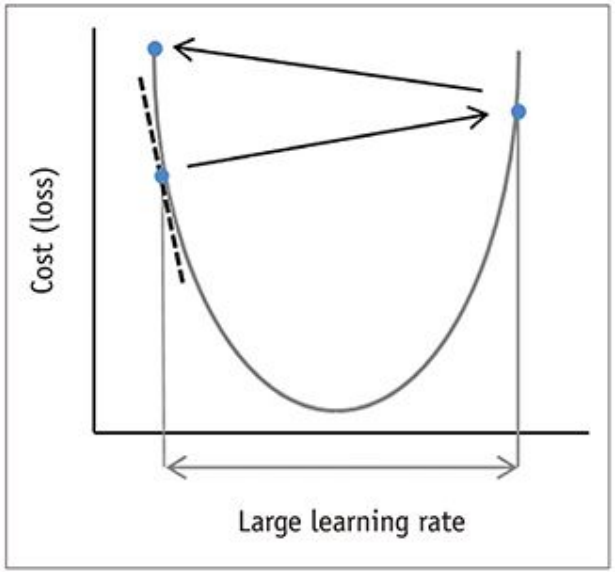
The gradient descent algorithm can be very sensitive to the value of learning rate:



Good 😊

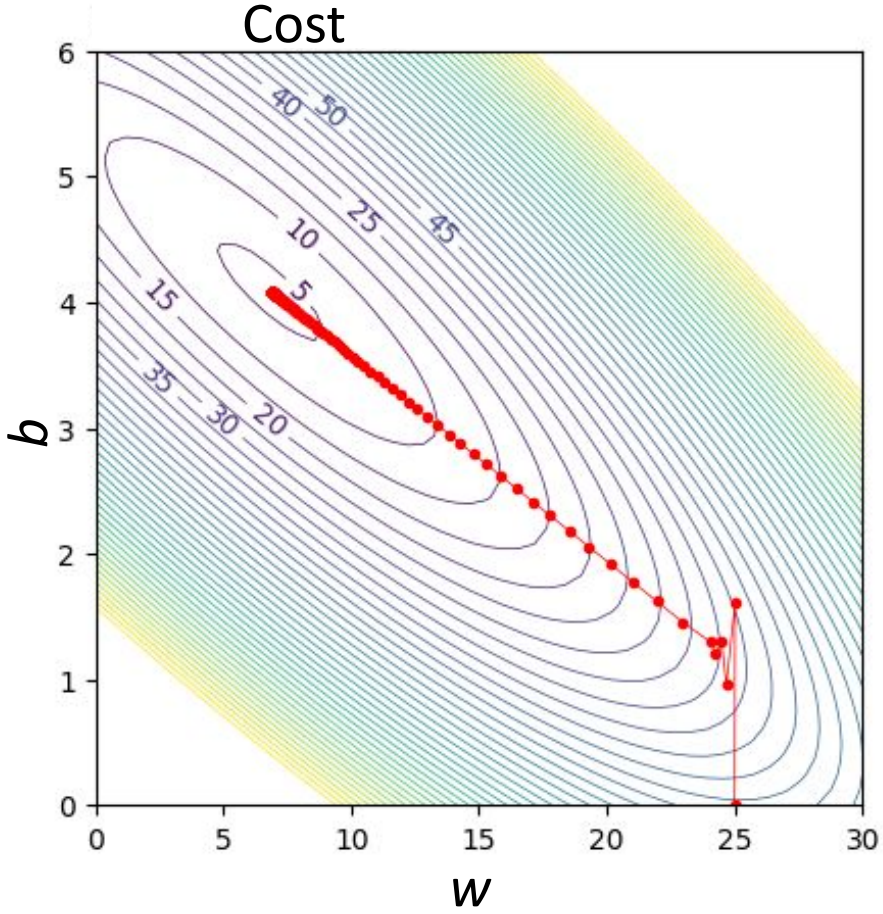


Bad 😑

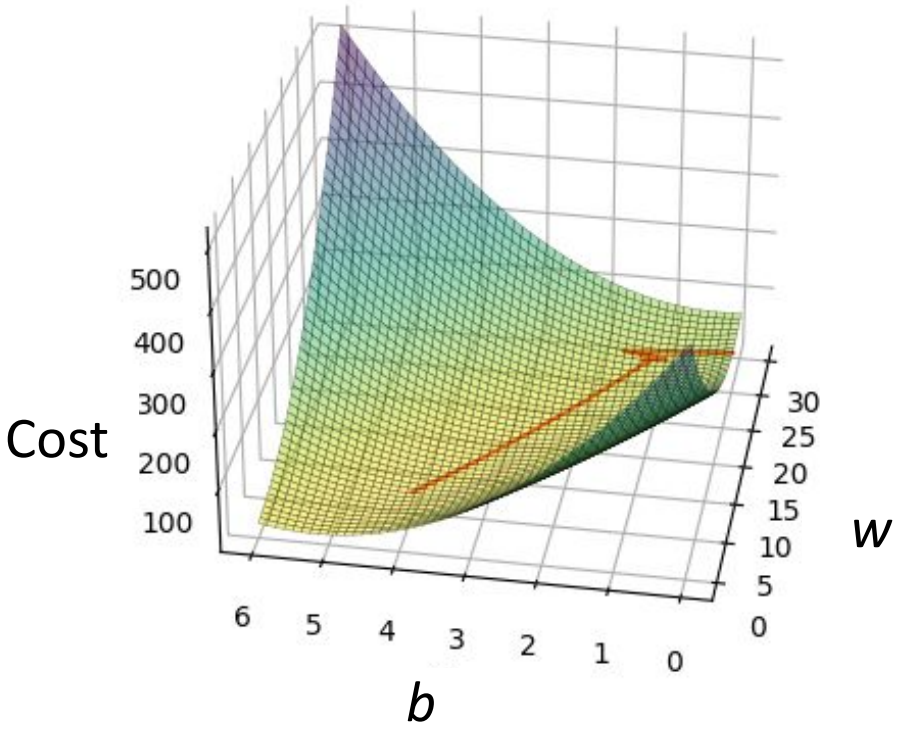


Ugly 😱

Gradient Descent “trajectory”



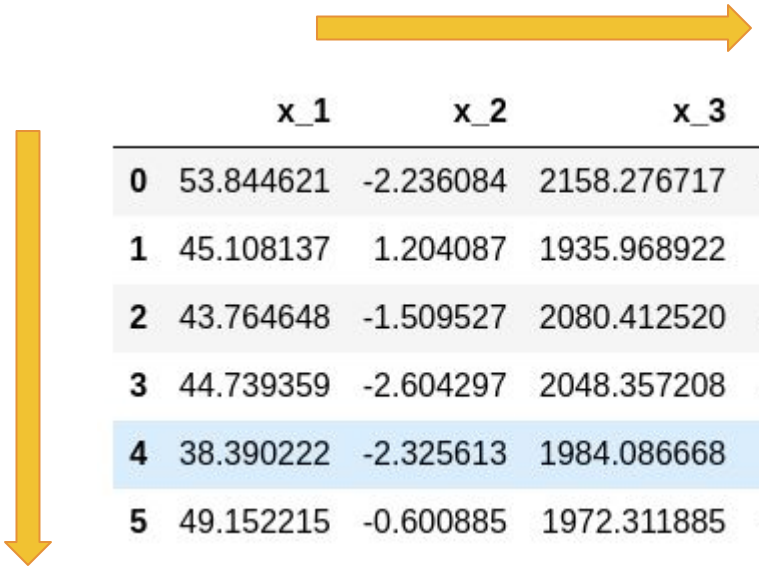
Convergence at the minimum:



Multivariate Linear Regression

Not one, but several input variables:

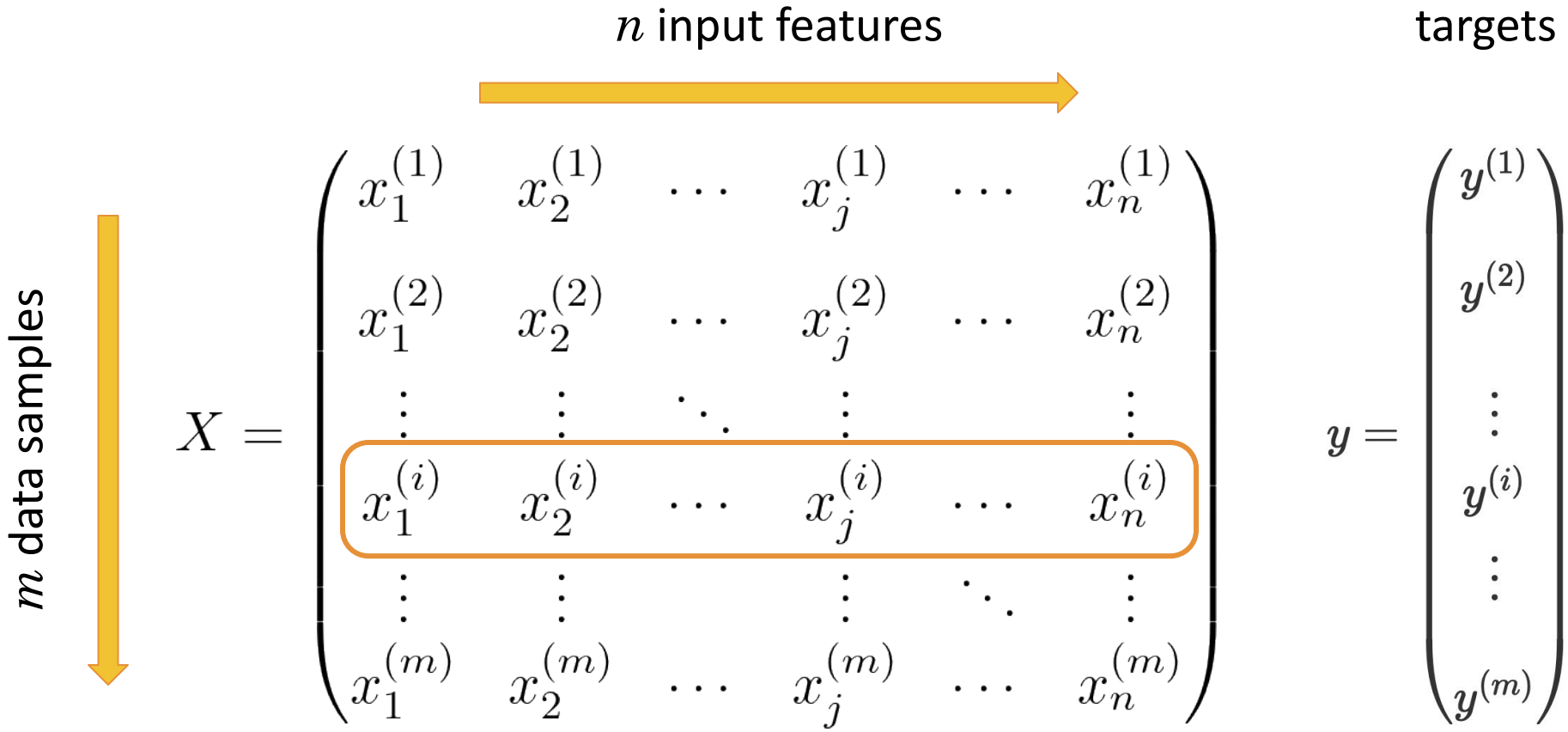
n input features



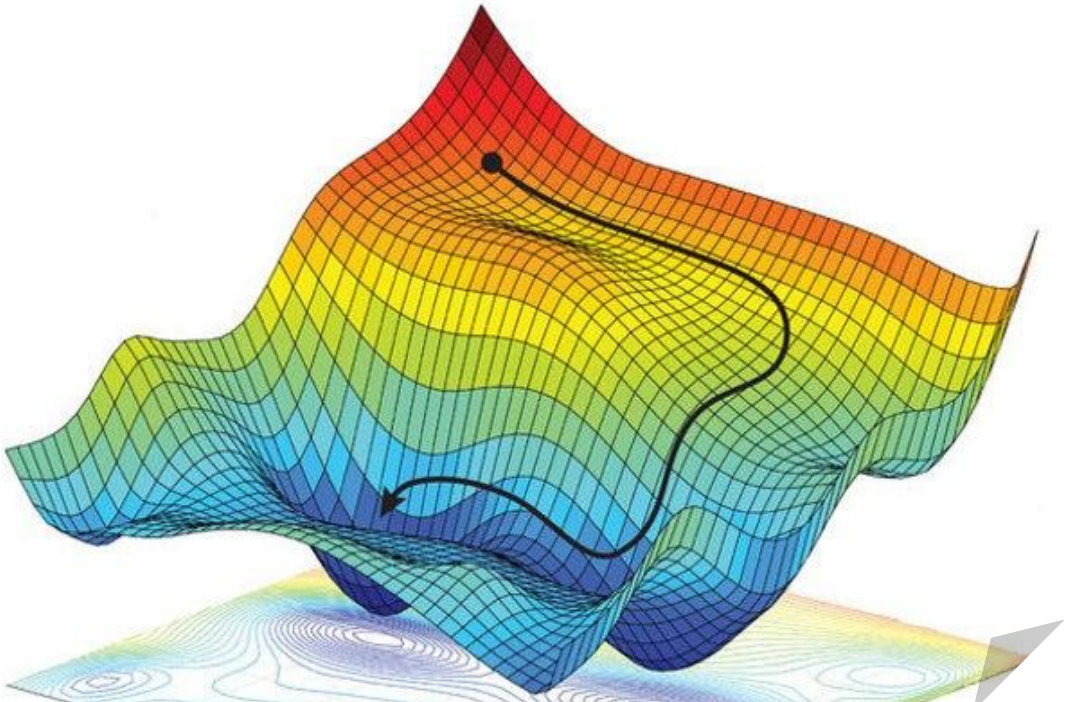
m data samples

	x_1	x_2	x_3	x_4
0	53.844621	-2.236084	2158.276717	-0.035128
1	45.108137	1.204087	1935.968922	0.038599
2	43.764648	-1.509527	2080.412520	-0.080645
3	44.739359	-2.604297	2048.357208	-0.040369
4	38.390222	-2.325613	1984.086668	0.243353
5	49.152215	-0.600885	1972.311885	-0.034590

Multivariate Linear Regression



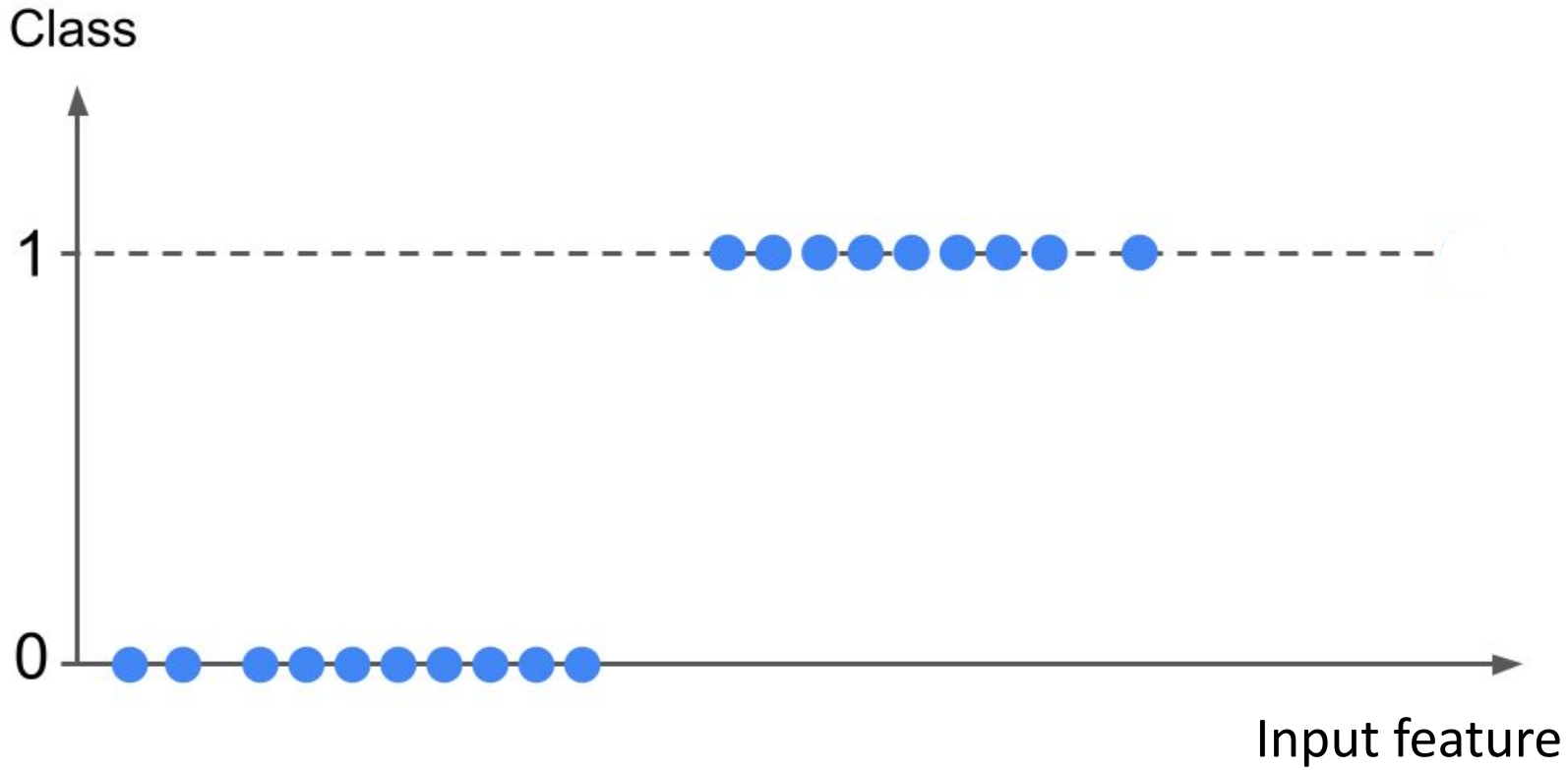
You okay?



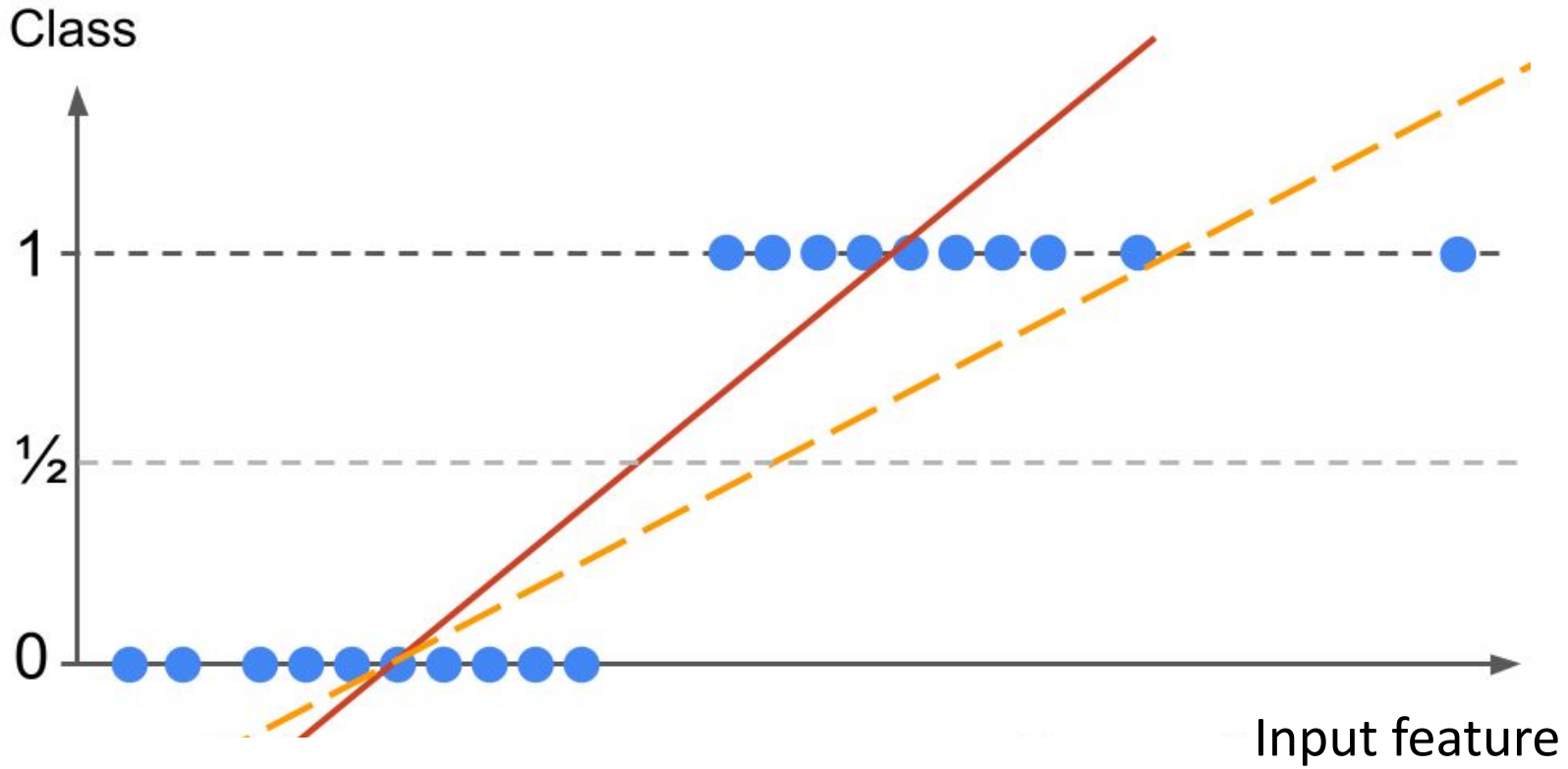
Logistic Regression

We classify

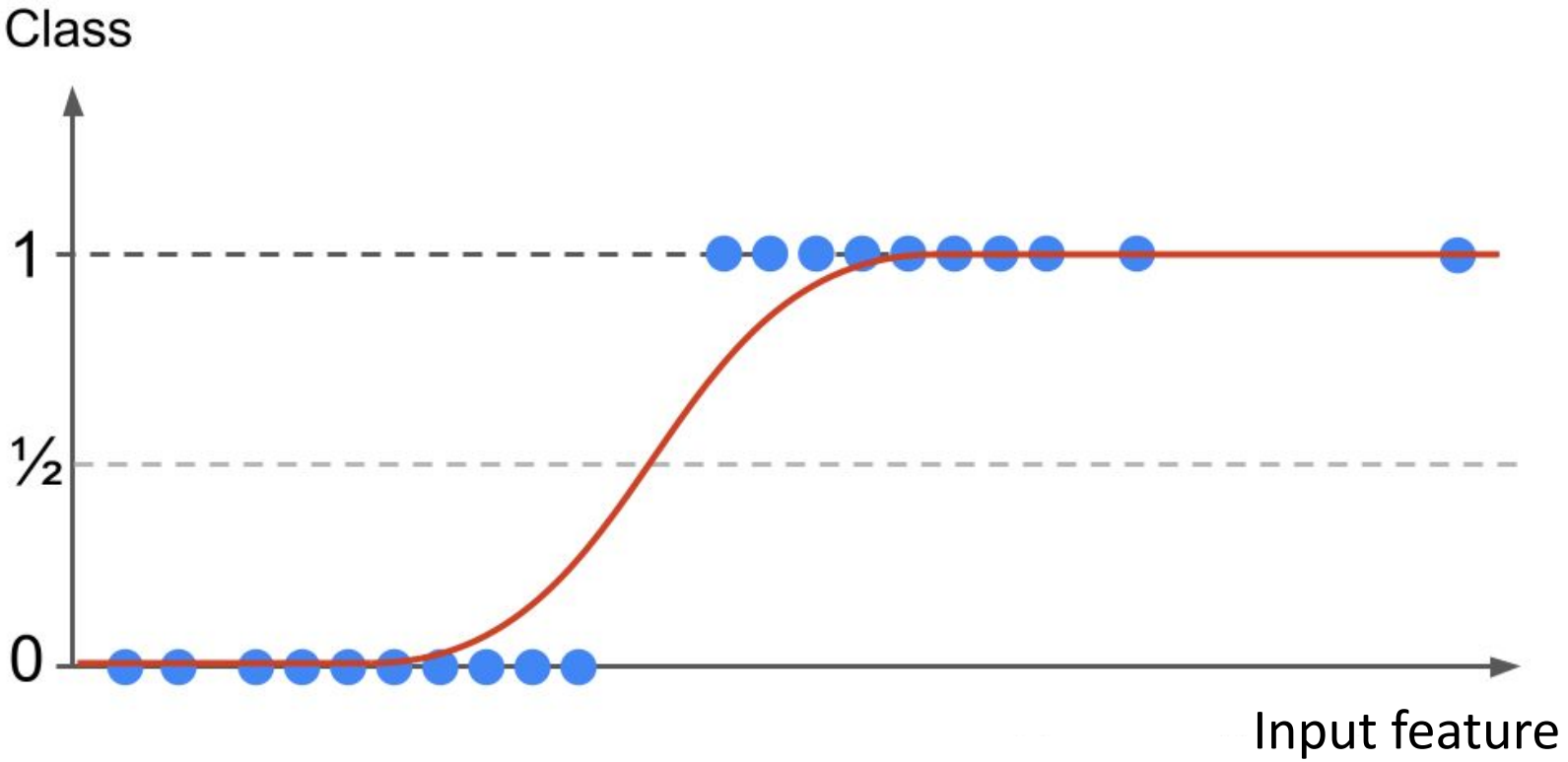
Warm up in 1 dimension



Linear Regression tools: problem!



A better mapping



The logistic function

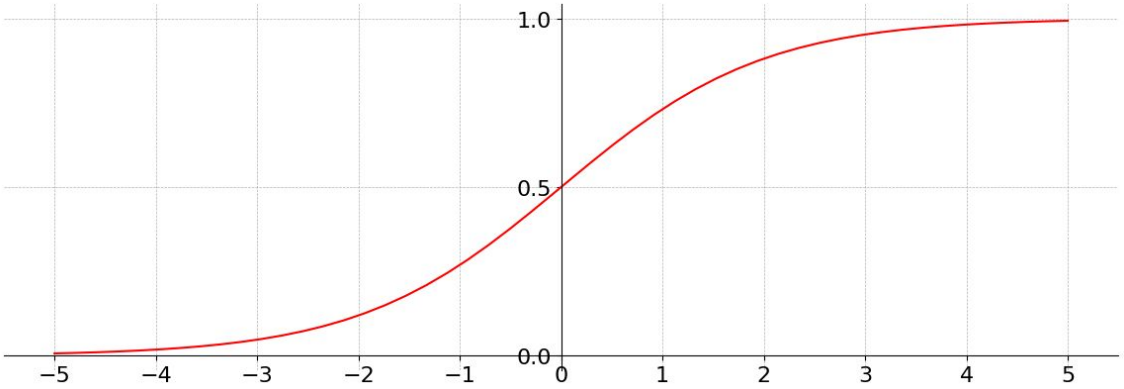
Definition Logistic function (sigmoid) is in the form:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Good for classification:

Mapping function is bounded between 0 and 1

Output can be interpreted as a probability.

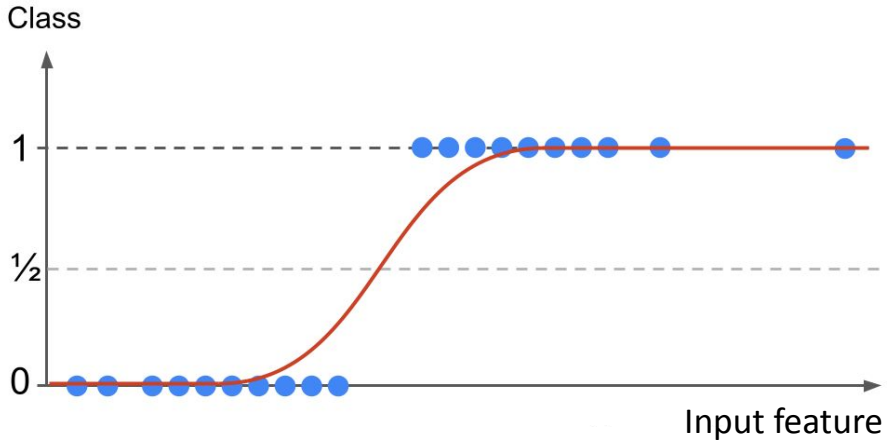


But how to classify?

Decision making

How to go from $h(x)$ $\rightarrow$ "the cassava leaf is predicted as class k"

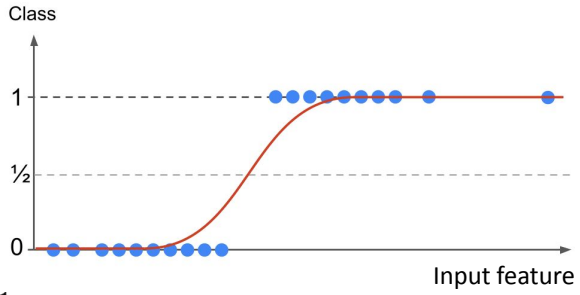
A **decision boundary** is a defined threshold (line, plane, etc) created by classifiers to discriminate between the different classes.



In our case: if $h(x) < 0.5 \Rightarrow$ leaf predicted as healthy

if $h(x) \geq 0.5 \Rightarrow$ leaf predicted as diseased

Summary Logistic Regression



Classification problems output(s) a **categorical variable**.

The hypothesis function is bounded by the (arbitrary) classes values.

The common hypothesis function is the **logistic/sigmoid**: $f(z) = \dfrac{1}{1+e^{-z}}$

The **decision boundary** is a threshold on the sigmoid output.

$$\sum_{j=1}^{n} w_j x_j + b$$

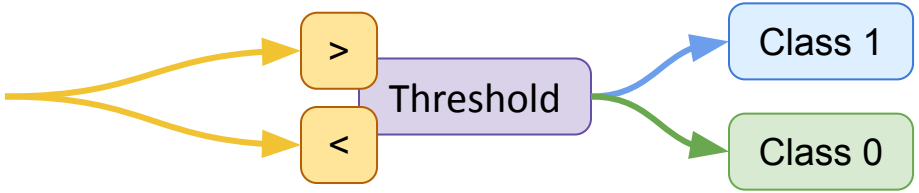
The standard cost function for binary classifiers is the **cross-entropy loss function**:

$$\text{Cost} = -\frac{1}{m}\sum_{i=1}^{m}\left[\; y^{(i)} \log(\; h(x^{(i)})) \;+\; (1-y^{(i)}) \log(1-\;\; h(x^{(i)}))\;\right]$$

Gradient descent gives optimized values of the parameters w_j and b

To predict:

$$y^{\text{pred}} = \frac{1}{1+\exp\left(-\sum w_j x_j^{\text{new}} + b\right)}$$



Neural Networks

Motivations

Suppose the data look like this:

Adding higher degree polynomials in the hypothesis function?

$$h(x) = w_{12} x_1 x_2 + w_{22} x_2^2 + w_{11} x_1^2 + b$$

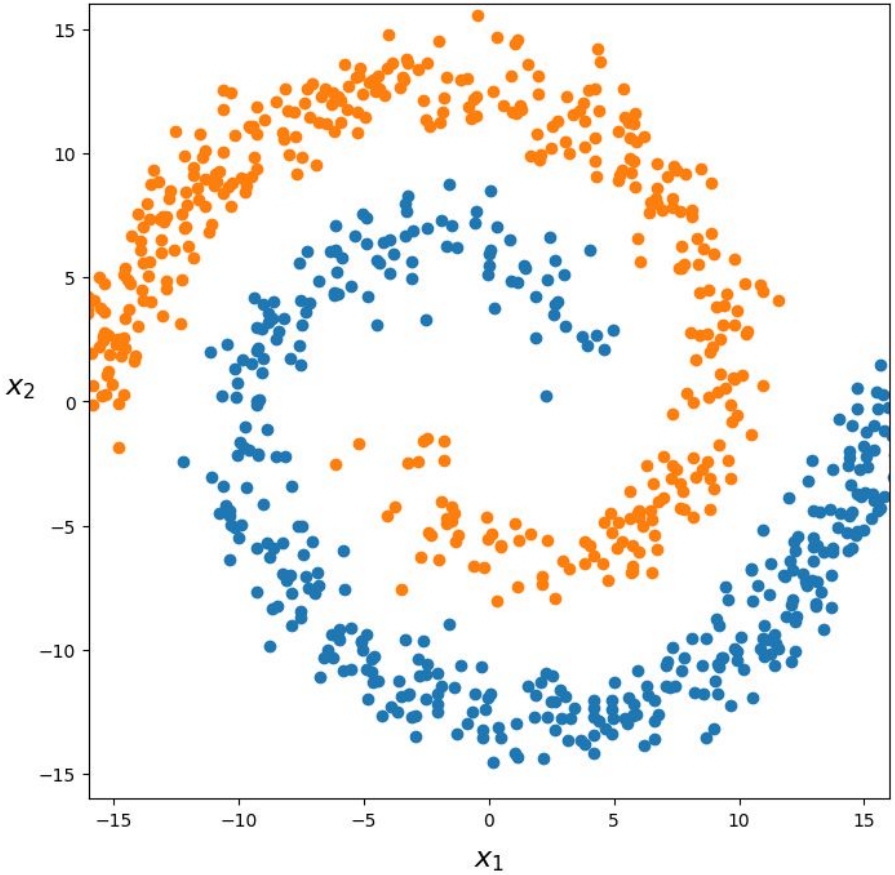
Scales like $O(\frac{1}{2} n^2)$

10 input features ⇒ 50 terms

100 input features ⇒ 5000 terms!

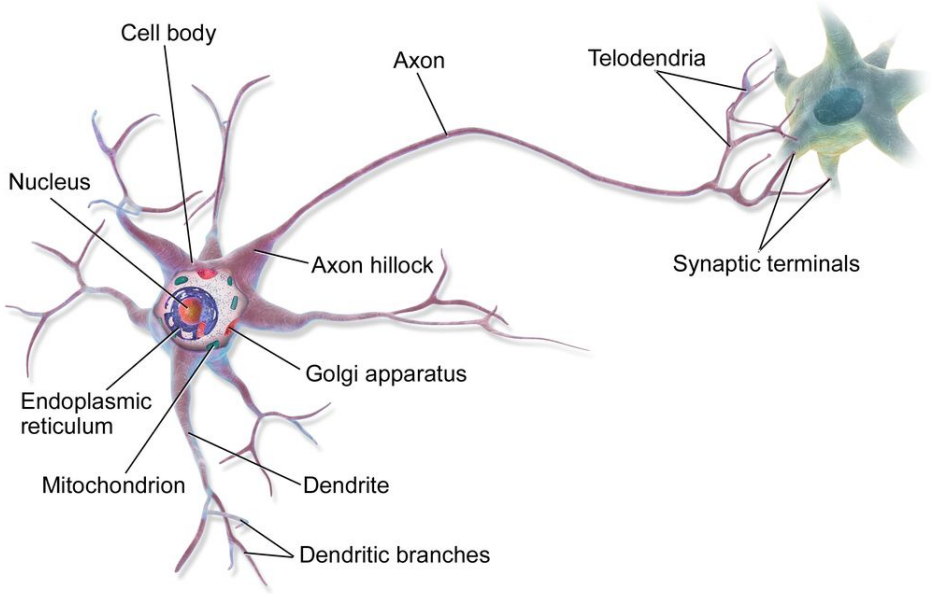
With cubic terms: scales like $O(n^3)$

IMPRACTICAL!



Introducing Neural Networks

An Artificial Neural Network is a Machine Learning Model inspired by the networks of biological neurons in animal brains.



The Artificial Neuron

Elementary computational unit of artificial neural networks

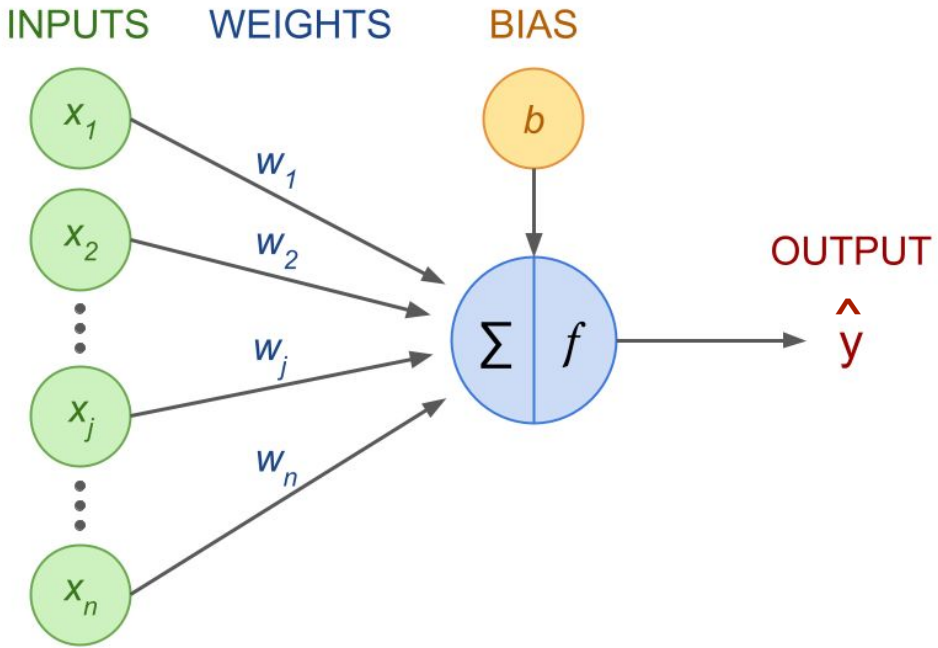
x_j : input nodes

w_j: weights

b : bias term

Σ: weighted sum

f: activation function



The maths inside an artificial neuron

The output is given by:

$$\hat{y} = f\left(\sum_{j=1}^{n} w_j x_j + b\right)$$

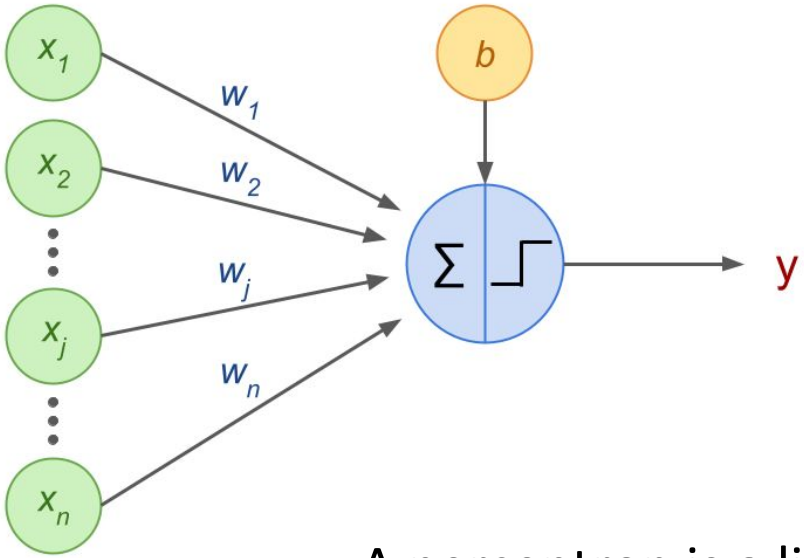
The **Activation Function** is a mathematical operation deciding whether the neuron's input to the network is important or not.

Returning a non-zero value means the neuron is "activated", or "fired."

The purpose of the activation function is to **introduce non-linearity** into the output of a neural network.

The Perceptron

Neural network made of a single neuron called Threshold Logic Unit (TLU) or Linear Threshold Unit (LTU), where the activation function is a step function.

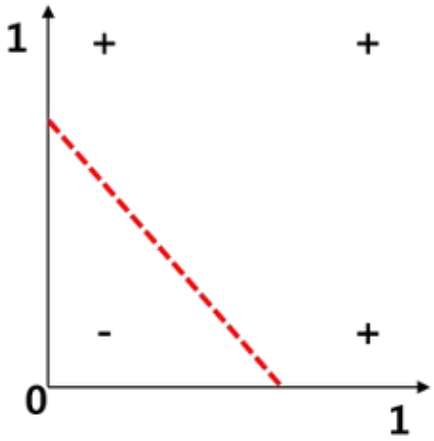


A perceptron is a linear binary classifier.

Perceptron and basic logical computations

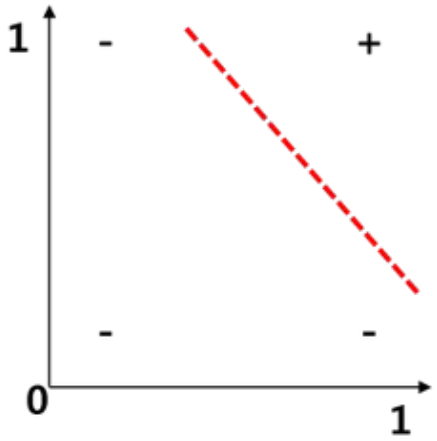
Logical Gates

OR



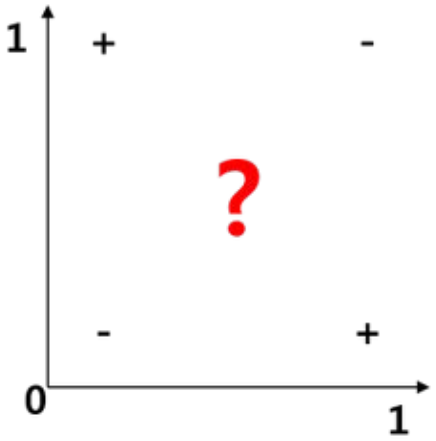
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

AND



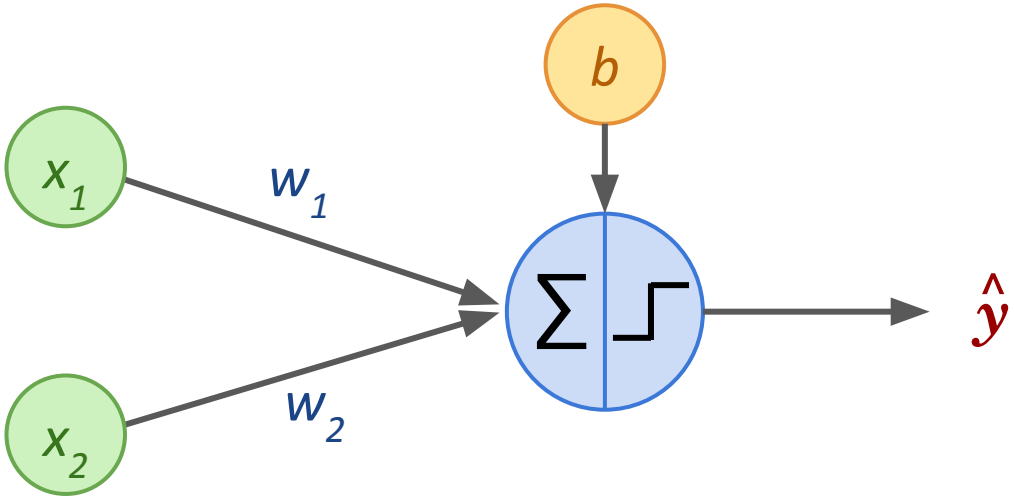
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

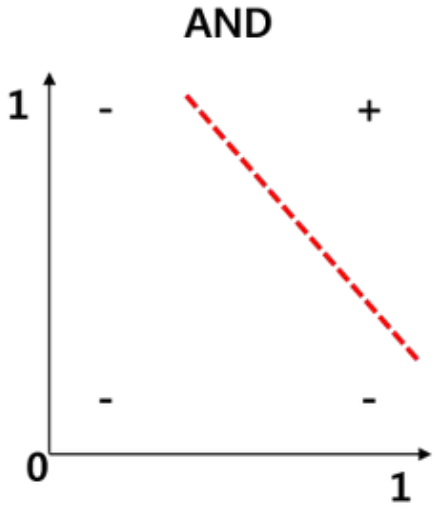
XOR



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Perceptron to solve “AND”



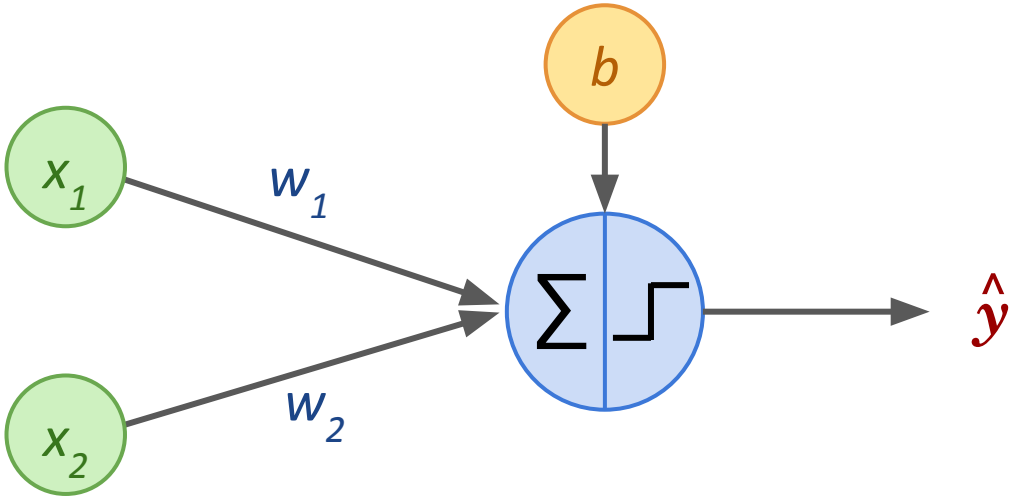


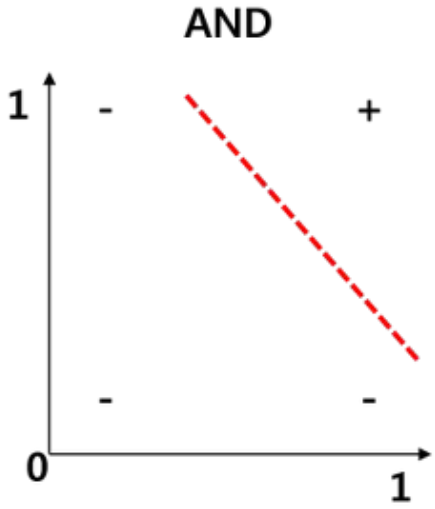
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Exercise:

What would be the weights w_1 , w_2 and bias b to encode the **AND** function? (using the Heaviside step function)

Perceptron to solve “AND”





x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Answer:

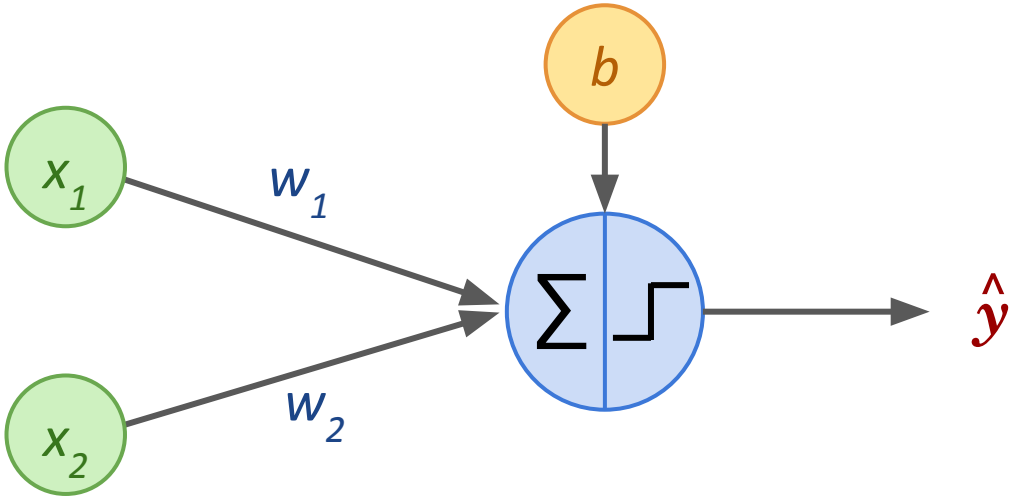
$b = -1$

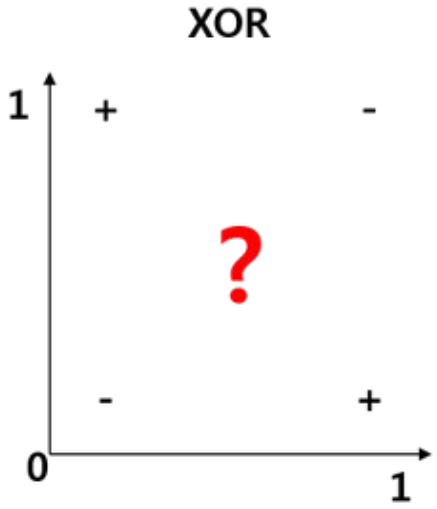
$w_1 = 0.5$

$w_2 = 0.5$

one possibility!

Perceptron to solve “XOR”?





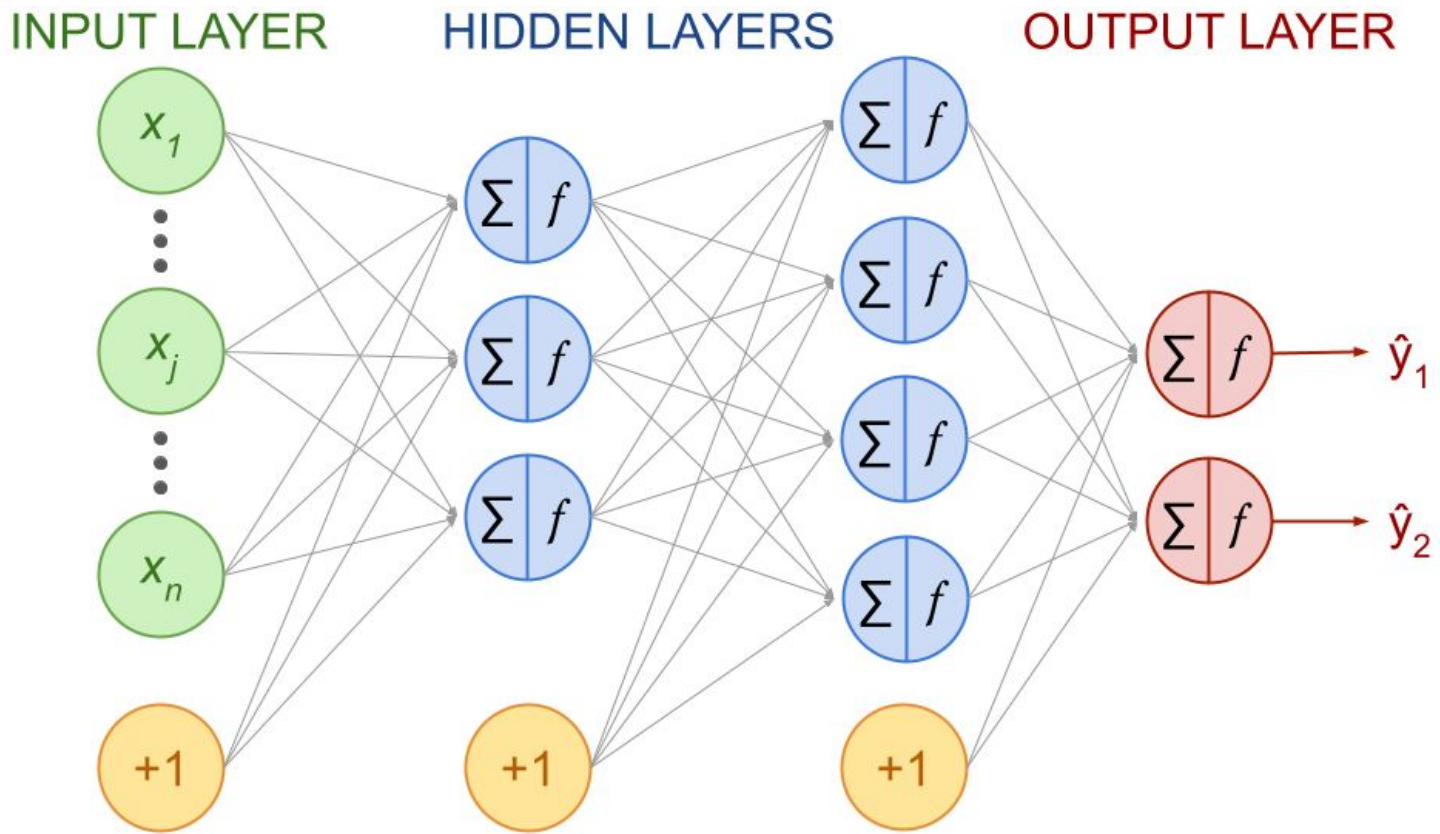
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Exercise:

What would be the weights w_1 , w_2 and bias b to encode the **XOR** function? (using the Heaviside step function)

Let’s build a network

Fully connected, feedforward neural network



Feedforward Propagation

Feedforward Propagation

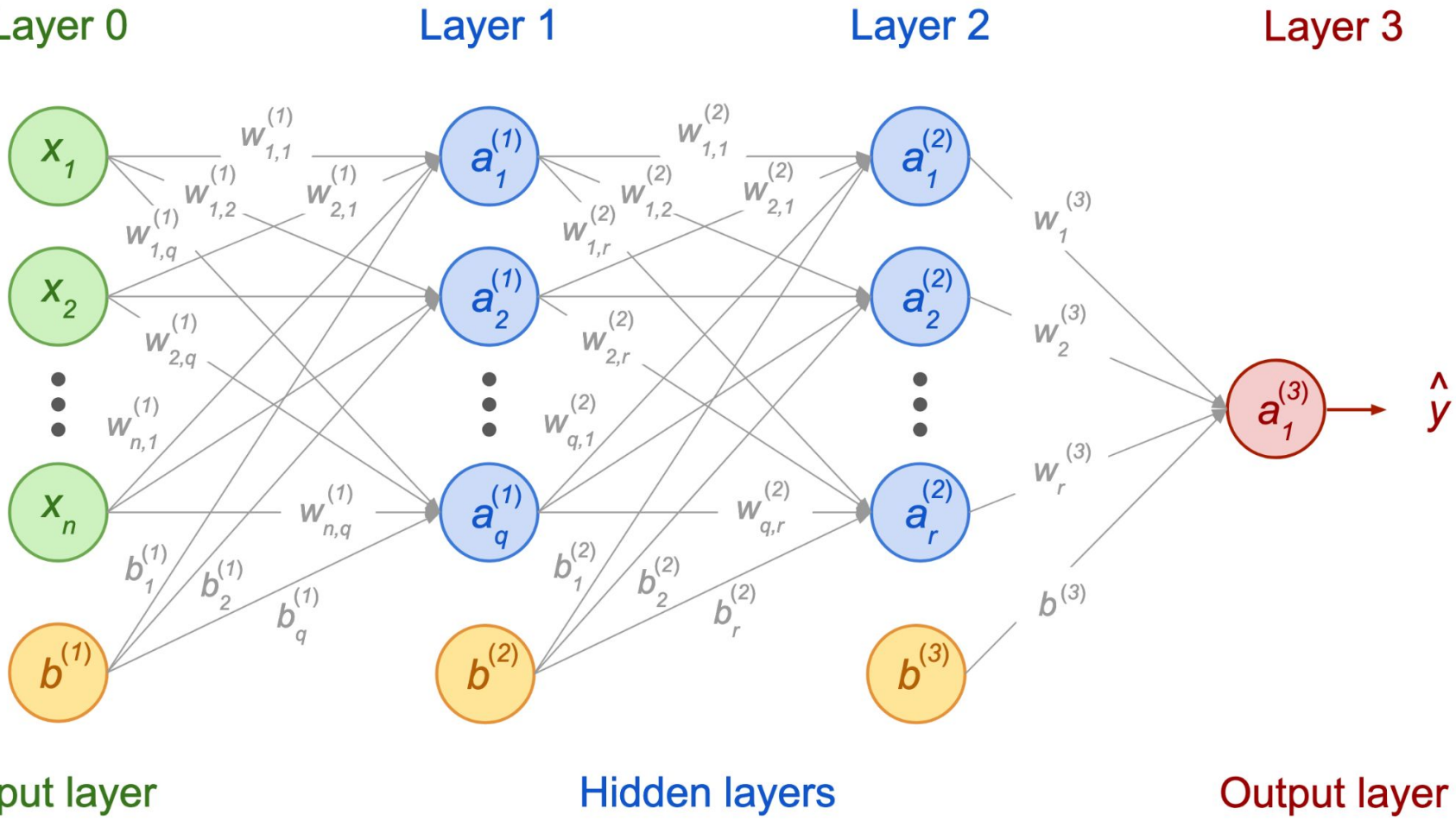
The **Feedforward Propagation**, also called **Forward Pass**, is the process consisting of computing all network nodes' output values, starting with the first hidden layer until the last output layer, using at start either a subset or the entire dataset samples.

Output: a list of predictions for each instance in dataset

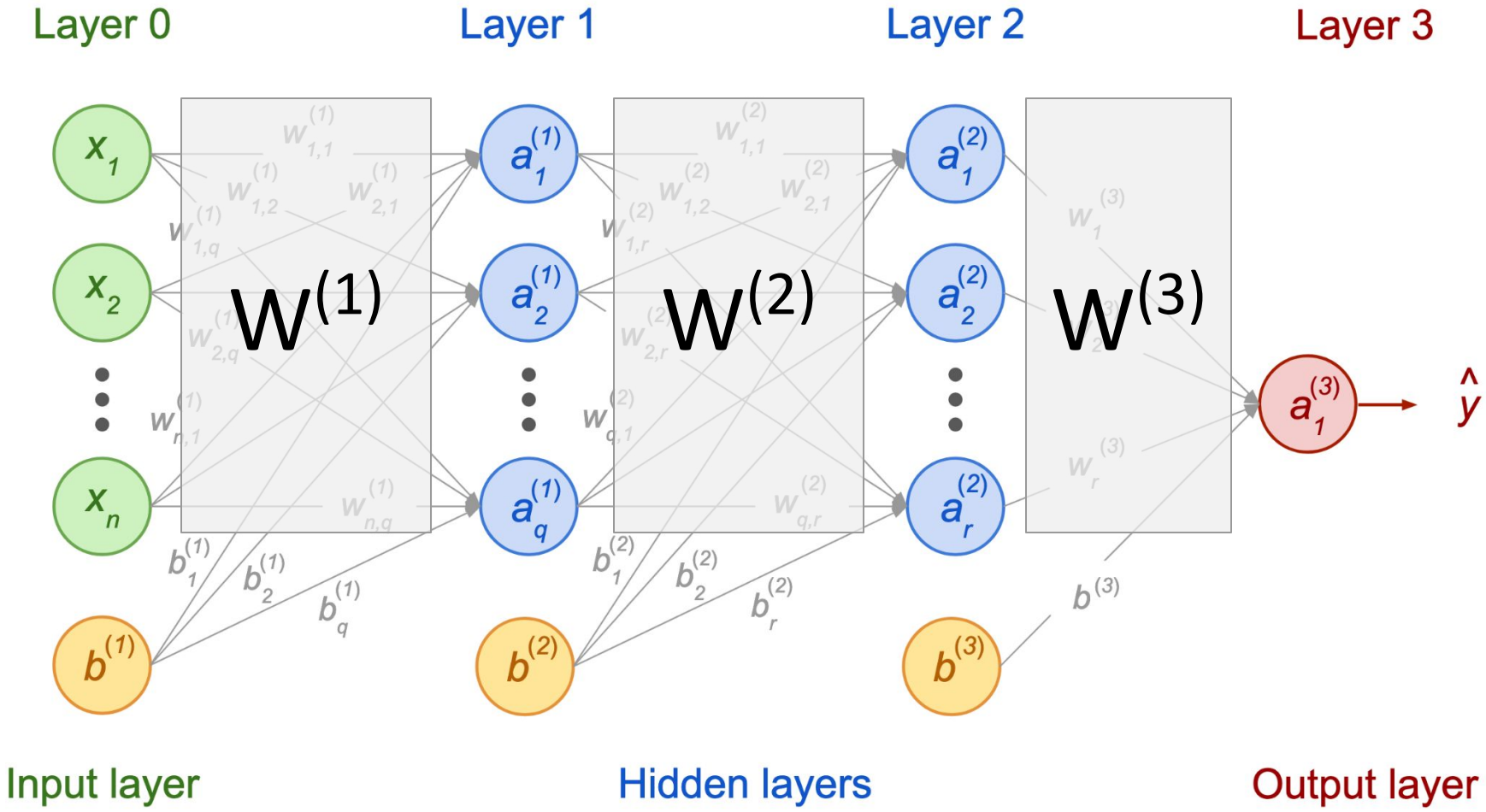
Key equation:

$$\hat{y} = f\left(\sum_{j=1}^{n} w_j x_j + b\right)$$

Notations


Layer 0
Layer 1
Layer 2
Layer 3
x_1
x_2
x_n
$b^{(1)}$
$a_1^{(1)}$
$a_2^{(1)}$
$a_q^{(1)}$
$b^{(2)}$
$a_1^{(2)}$
$a_2^{(2)}$
$a_r^{(2)}$
$b^{(3)}$
$a_1^{(3)}$
$\hat{y}$
$w_{1,1}^{(1)}$
$w_{1,2}^{(1)}$
$w_{1,q}^{(1)}$
$w_{2,1}^{(1)}$
$w_{2,q}^{(1)}$
$w_{n,1}^{(1)}$
$w_{n,q}^{(1)}$
$b_1^{(1)}$
$b_2^{(1)}$
$b_q^{(1)}$
$w_{1,1}^{(2)}$
$w_{1,2}^{(2)}$
$w_{1,r}^{(2)}$
$w_{2,1}^{(2)}$
$w_{2,r}^{(2)}$
$w_{q,1}^{(2)}$
$w_{q,r}^{(2)}$
$b_1^{(2)}$
$b_2^{(2)}$
$b_r^{(2)}$
$w_1^{(3)}$
$w_2^{(3)}$
$w_r^{(3)}$
$b^{(3)}$
Input layer
Hidden layers
Output layer

Notations


Layer 0
Layer 1
Layer 2
Layer 3
x_1
x_2
x_n
$W^{(1)}$
$W^{(2)}$
$W^{(3)}$
$a_1^{(1)}$
$a_2^{(1)}$
$a_q^{(1)}$
$a_1^{(2)}$
$a_2^{(2)}$
$a_r^{(2)}$
$a_1^{(3)}$
$\hat{y}$
$b^{(1)}$
$b^{(2)}$
$b^{(3)}$
Input layer
Hidden layers
Output layer

Notations

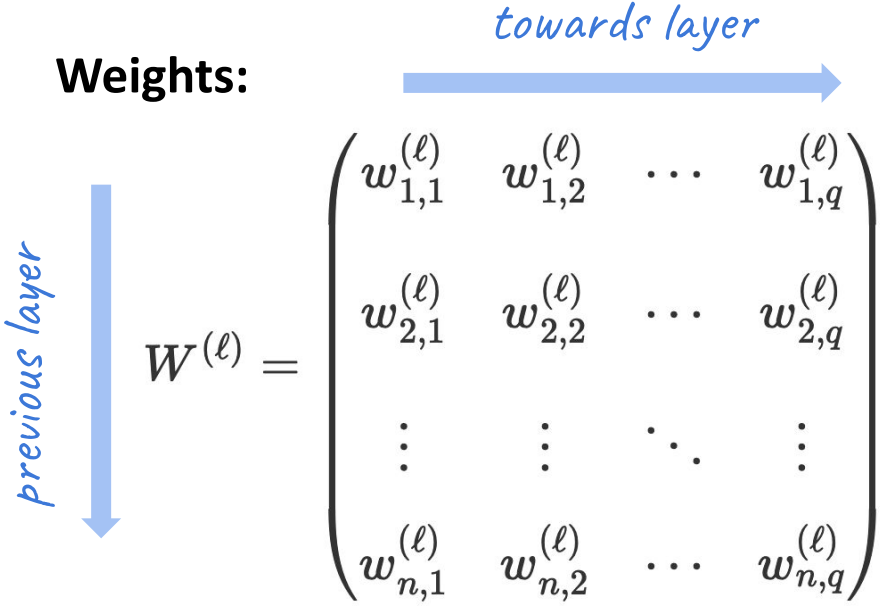
Inputs:

$$\boldsymbol{x}^{(i)} = \begin{pmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_n^{(i)} \end{pmatrix}$$

Activation units:

$$\boldsymbol{a}^{(i,\,\ell)} = \begin{pmatrix} a_1^{(i,\,\ell)} \\ a_2^{(i,\,\ell)} \\ \vdots \\ a_q^{(i,\,\ell)} \end{pmatrix}$$

Weights:



Biases:

$$\boldsymbol{b}^{(\ell)} = \begin{pmatrix} b_1^{(\ell)} \\ b_2^{(\ell)} \\ \vdots \\ b_q^{(\ell)} \end{pmatrix}$$

Q1: Which entities are sample-dependent?

Q2: Where are the model parameters?

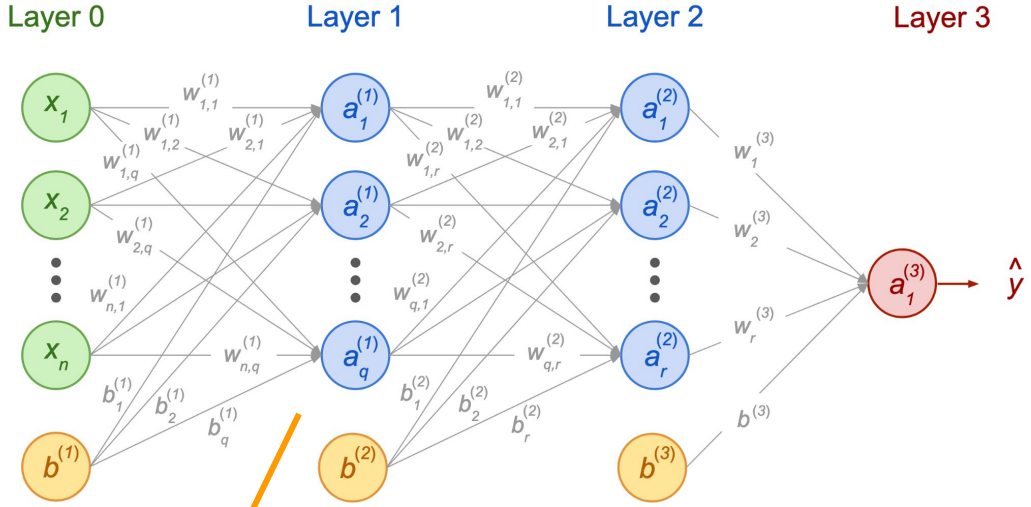
Notations

Inputs:

$$\boldsymbol{x}^{(i)} = \begin{pmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_n^{(i)} \end{pmatrix}$$

Activation units:

$$\boldsymbol{a}^{(i,\,\ell)} = \begin{pmatrix} a_1^{(i,\,\ell)} \\ a_2^{(i,\,\ell)} \\ \vdots \\ a_q^{(i,\,\ell)} \end{pmatrix},$$



Weights:

towards layer

previous layer

$$W^{(\ell)} = \begin{pmatrix} w_{1,1}^{(\ell)} & w_{1,2}^{(\ell)} & \cdots & w_{1,q}^{(\ell)} \\ w_{2,1}^{(\ell)} & w_{2,2}^{(\ell)} & \cdots & w_{2,q}^{(\ell)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n,1}^{(\ell)} & w_{n,2}^{(\ell)} & \cdots & w_{n,q}^{(\ell)} \end{pmatrix}$$

$$a_1^{(1)} = f\left(w_{1,1}^{(1)}\, x_1 + w_{2,1}^{(1)}\, x_2 + \cdots + w_{n,1}^{(1)}\, x_n + b_1^{(1)} \right)$$

$$a_2^{(1)} = f\left(w_{1,2}^{(1)}\, x_1 + w_{2,2}^{(1)}\, x_2 + \cdots + w_{n,2}^{(1)}\, x_n + b_2^{(1)} \right)$$

$$\vdots$$

$$a_q^{(1)} = f\left(w_{1,q}^{(1)}\, x_1 + w_{2,q}^{(1)}\, x_2 + \cdots + w_{n,q}^{(1)}\, x_n + b_q^{(1)} \right)$$

Feedforward: all units in first hidden layer (1)

Matrix form (expanded)

$$\boldsymbol{a}^{(i,\,1)} = f\left[\begin{pmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} & \cdots & w_{1,q}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} & \cdots & w_{2,q}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n,1}^{(1)} & w_{n,2}^{(1)} & \cdots & w_{n,q}^{(1)} \end{pmatrix}^{\top} \begin{pmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \cdots \\ x_n^{(i)} \end{pmatrix} + \begin{pmatrix} b_1^{(1)} \\ b_2^{(1)} \\ \cdots \\ b_q^{(1)} \end{pmatrix}\right]$$

More compact as: $$\boldsymbol{a}^{(i,\,1)} = f\left[\left(W^{(1)}\right)^{\top} \boldsymbol{x}^{(i)} + \boldsymbol{b}^{(1)}\right]$$

Feedforward: all units in second hidden layer (2)

Matrix form (expanded)

$$\boldsymbol{a}^{(i,\,2)} = f\left[\begin{pmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & \cdots & w_{1,r}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & \cdots & w_{2,r}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{q,1}^{(2)} & w_{q,2}^{(2)} & \cdots & w_{q,r}^{(2)} \end{pmatrix}^{\top} \begin{pmatrix} a_1^{(i,\,1)} \\ a_2^{(i,\,1)} \\ \cdots \\ a_q^{(i,\,1)} \end{pmatrix} + \begin{pmatrix} b_1^{(2)} \\ b_2^{(2)} \\ \cdots \\ b_r^{(2)} \end{pmatrix}\right]$$

More compact as: $\boldsymbol{a}^{(i,\,2)} = f\left[\left(W^{(2)}\right)^{\top} \boldsymbol{a}^{(i,\,1)} + \boldsymbol{b}^{(2)}\right]$

General Rule for Feedforward propagation

Renaming the inputs as a “layer zero”:

$$\boldsymbol{x}^{(i)} = \begin{pmatrix} x_1^{(i,\,0)} \\ x_2^{(i,\,0)} \\ \cdots \\ x_n^{(i,\,0)} \end{pmatrix} = \begin{pmatrix} a_1^{(i,\,0)} \\ a_2^{(i,\,0)} \\ \cdots \\ a_n^{(i,\,0)} \end{pmatrix} = \boldsymbol{a}^{(i,\,0)}$$

General equation for layer $\boldsymbol{\ell}$:

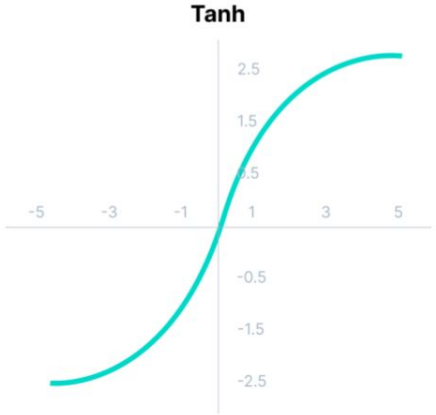
$$\boldsymbol{a}^{(i,\,\ell)} = f\left[\, \left(W^{(\ell)}\right)^{\top} \boldsymbol{a}^{(i,\,\ell-1)} + \boldsymbol{b}^{(\ell)} \,\right]$$

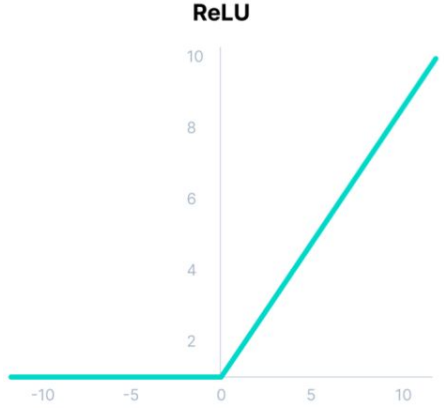
Activation functions' properties

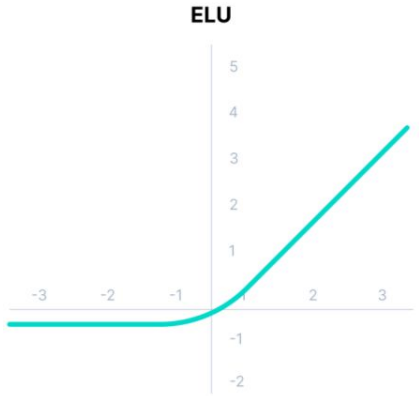
Differentiability better if continuously differentiable

Range if bounded ⇒ more stable during gradient descent

Non-linearity essential for the neural network to learn





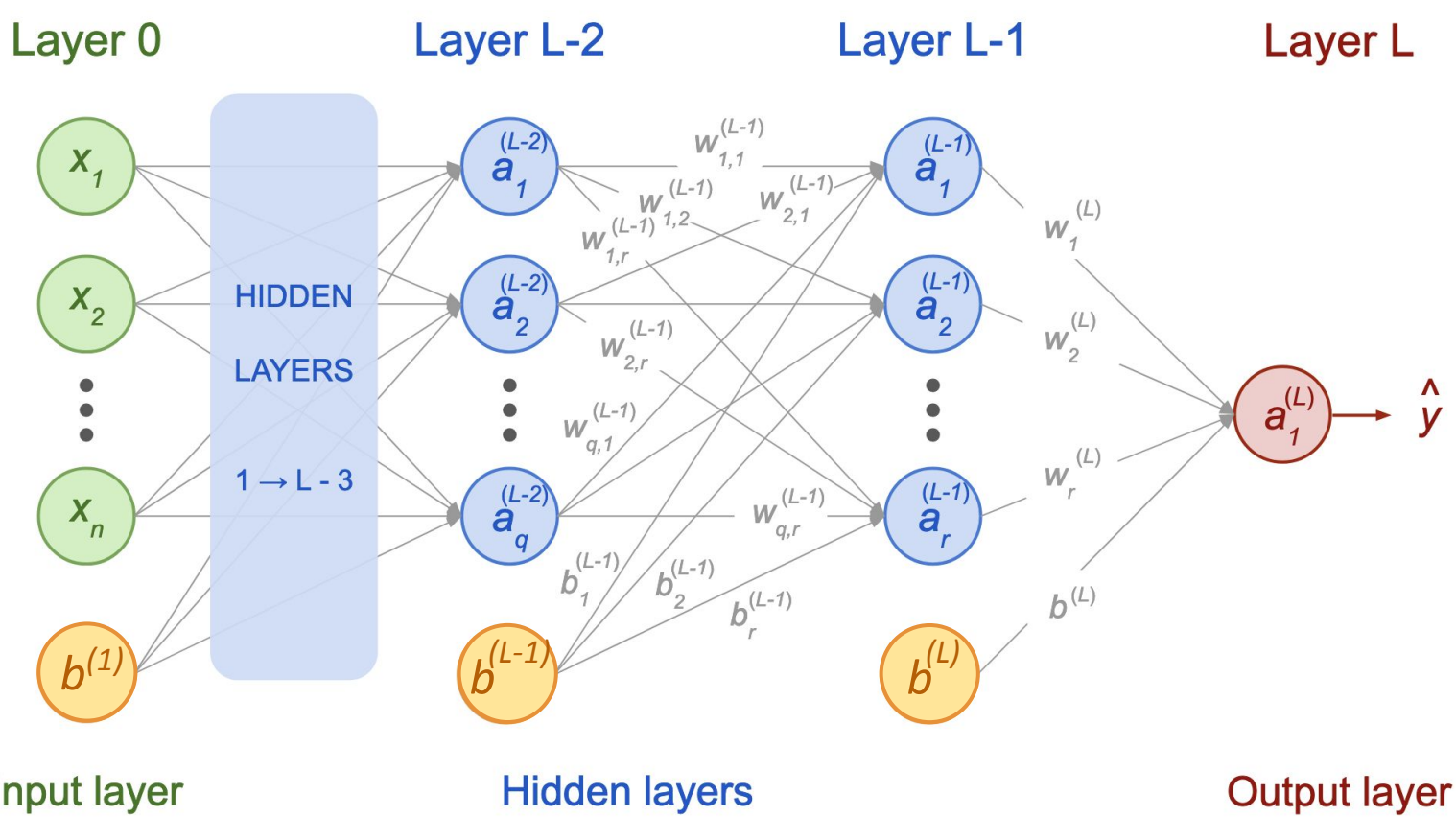


Summary Forward Propagation

- Process of **computing all activation units** of a NN.
- Leads to **prediction** at the last layer(s)
- Predictions will be **compared** to the **observed values**.
- Feedforward propagation is a **step in the training** of a neural network.

Backward Propagation

Going backwards



Backpropagation

Backpropagation, or **backward propagation of errors**, is an algorithm working from the output nodes to the input nodes of a neural network using the chain rule to compute how much each activation unit contributed to the overall error.

Backpropagation automatically computes error gradients to then repeatedly adjust all weights and biases to reduce the overall error.

How Neural Networks are trained

Main Steps

What do we get?

Step 0: Initialization

Step 1: Forward Propagation → list of predictions $\hat{\boldsymbol{y}}^{(i)}$

Step 2: Back Propagation → list of errors $\boldsymbol{\delta}^{(i,\,\ell)}$

→ cost derivatives $\dfrac{\partial\ \text{Cost}}{\partial \boldsymbol{W}^{(\ell)}}$ $\dfrac{\partial\ \text{Cost}}{\partial \boldsymbol{b}^{(\ell)}}$

Step 3: Gradient Descent

→ update weights: $W^{(\ell)} \leftarrow W^{(\ell)} - \alpha \dfrac{\partial\ \text{Cost}}{\partial W^{(\ell)}}$

→ update biases: $\boldsymbol{b}^{(\ell)} \leftarrow \boldsymbol{b}^{(\ell)} - \alpha \dfrac{\partial\ \text{Cost}}{\partial \boldsymbol{b}^{(\ell)}}$

Repeat 1 → 3

Exit after $N^{\text{iterations}}$ or derivatives → 0

How to calculate the error?

$$\hat{\boldsymbol{y}}^{(i)} \longrightarrow \boldsymbol{\delta}^{(i,\,\ell)}$$

Loss Function & Cost Function

Loss Functions for Classification

Binary Cross-Entropy

$$L\left(\hat{y}^{(i)}, y^{(i)}\right) = -\left[\, y^{(i)} \log\left(\hat{y}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - \hat{y}^{(i)}\right) \,\right]$$

Associated cost:

$$J\left(\hat{y}, y\right) = -\frac{1}{m} \sum_{i=1}^{m} \left[\, y^{(i)} \log\left(\hat{y}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - \hat{y}^{(i)}\right) \,\right]$$

$h_{W,b}(x^{(i)})$ $h_{W,b}(x^{(i)})$

hypothesis no more linear here!

Loss and Cost functions

The **loss function** L quantifies the difference between the actual and predicted value for one sample instance.

The **cost function** J aggregates the differences of all instances of the dataset. It can have a regularization term.

Loss: $L\left(\hat{y}^{(i)}, y^{(i)}\right)$ ⇨ $\text{Cost} = \frac{1}{m}\sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) = \frac{1}{m}\sum L\left(\hat{\boldsymbol{y}}, \boldsymbol{y}\right)$

Categorical Cross-Entropy

If more than 2 classes:

Observations: $\boldsymbol{y}^{(i)} = (0, 1, 0, \cdots 0)$

Predictions: $\hat{\boldsymbol{y}}^{(i)} = (0.15, 0.68, \cdots, 0.03)$ with $\sum_{k=1}^{K} \hat{y}_k^{(i)} = 1$

Loss:

$$L\left(\hat{y}_k^{(i)}, y_k^{(i)} \right) = -\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{y}_k^{(i)} \right)$$

Cost:

$$J\left(\hat{\boldsymbol{y}}, \boldsymbol{y} \right) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{y}_k^{(i)} \right)$$

reduces to cross-entropy for $n = 2$

Chain rule refresher

For 3 functions such as: $k(x) = h(f(g(x)))$

Use the chain rule to express $k'(x)$

Chain rule refresher

For 3 functions: $k(x) = h(f(g(x)))$

Use the chain rule to express $k'(x)$

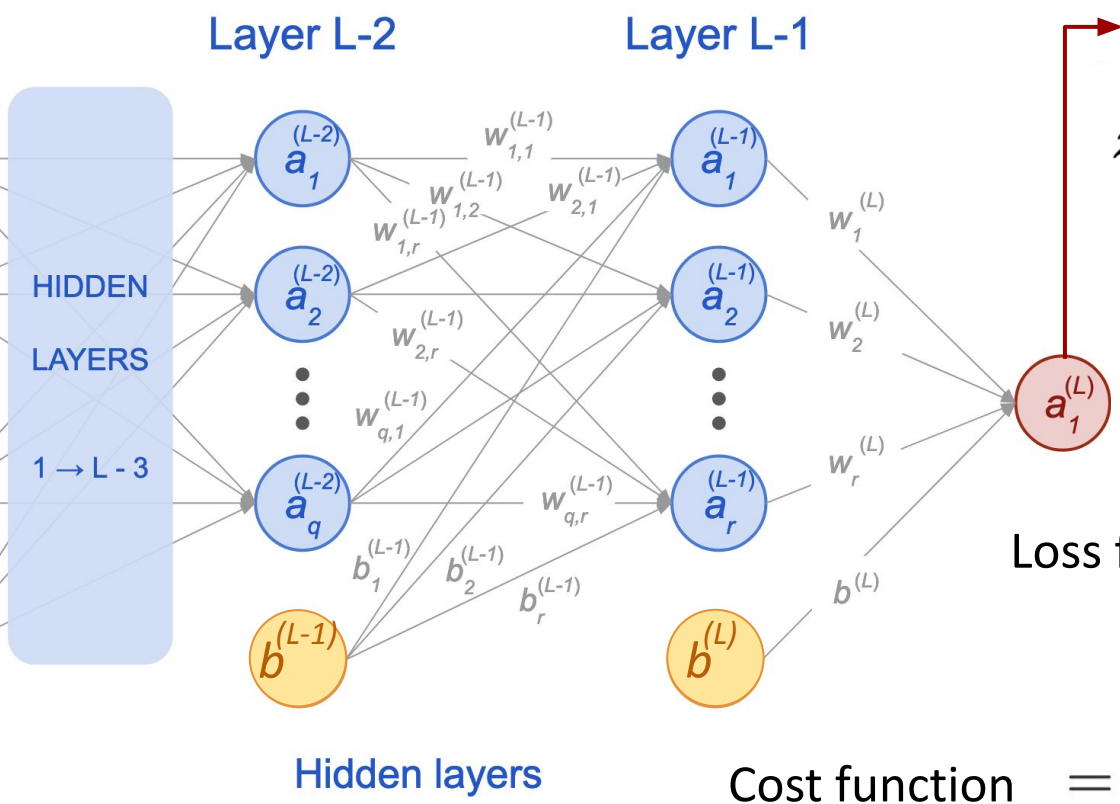
$$k'(x) = \frac{\mathrm{d}\, h(f(g(x)))}{\mathrm{d}\, f(g(x))} \cdot \frac{\mathrm{d}\, f(g(x))}{\mathrm{d}\, g(x)} \cdot \frac{\mathrm{d}\, g(x)}{\mathrm{d}\, x}$$

$$= h'(f(g(x))) \cdot f'(g(x)) \cdot g'(x)$$

three functions ⟶ three derivative terms

Main Steps of Backpropagation

From predictions to the cost



Last activation layer L = prediction $\hat{y}$

Weighted sum at last layer:

$$z^{(i,\,L)} = \left(W^{(L)}\right)^{\top} a^{(i,\,L-1)} + b^{(L)}$$

Activation: $a^{(i,\,L)} = f\left(z^{(i,\,L)}\right)$

Loss function: $L\left(f\left(z^{(i,\,L)}\right), y^{(i)}\right)$

target

Cost function $= \frac{1}{m}\sum_{i=1}^{m} L\left(f\left(z^{(i,\,L)}\right), y^{(i)}\right)$

prediction

From cost to partial derivatives

Reminder of the goal:

We want to know how the cost varies with respect to?

- the weight matrices $\boldsymbol{W}$
- the biases $\boldsymbol{b}$

Mathematical rephrasing of the goal:

Find parameters $\boldsymbol{W}$ and $\boldsymbol{b}$ minimizing the overall error: $\min_{\boldsymbol{W},\boldsymbol{b}} \text{Cost}(\boldsymbol{W},\boldsymbol{b})$

Need to compute: Partial derivatives $\dfrac{\partial \text{Cost}(\boldsymbol{W},\boldsymbol{b})}{\partial \boldsymbol{W}}$ $\dfrac{\partial \text{Cost}(\boldsymbol{W},\boldsymbol{b})}{\partial \boldsymbol{b}}$

Where to start & how?

From last layer L (prediction) and applying the chain rule:

$$\text{Cost} = \frac{1}{m}\sum_{i=1}^{m} L\left(f\left(\boldsymbol{z}^{(i,\,L)}\right), \boldsymbol{y}^{(i)}\right)$$

← 3 functions

Last Layer

From the cost's equation, applying the chain rule:

$$\frac{\partial \text{ Cost}}{\partial W^{(L)}} = \frac{1}{m}\sum_{i=1}^{m} \frac{\partial L(f(\boldsymbol{z}^{(i,L)}), \boldsymbol{y}^{(i)})}{\partial W^{(L)}}$$

$$= \frac{1}{m}\sum_{i=1}^{m} \square \; ? \; \square \; ? \; \square$$

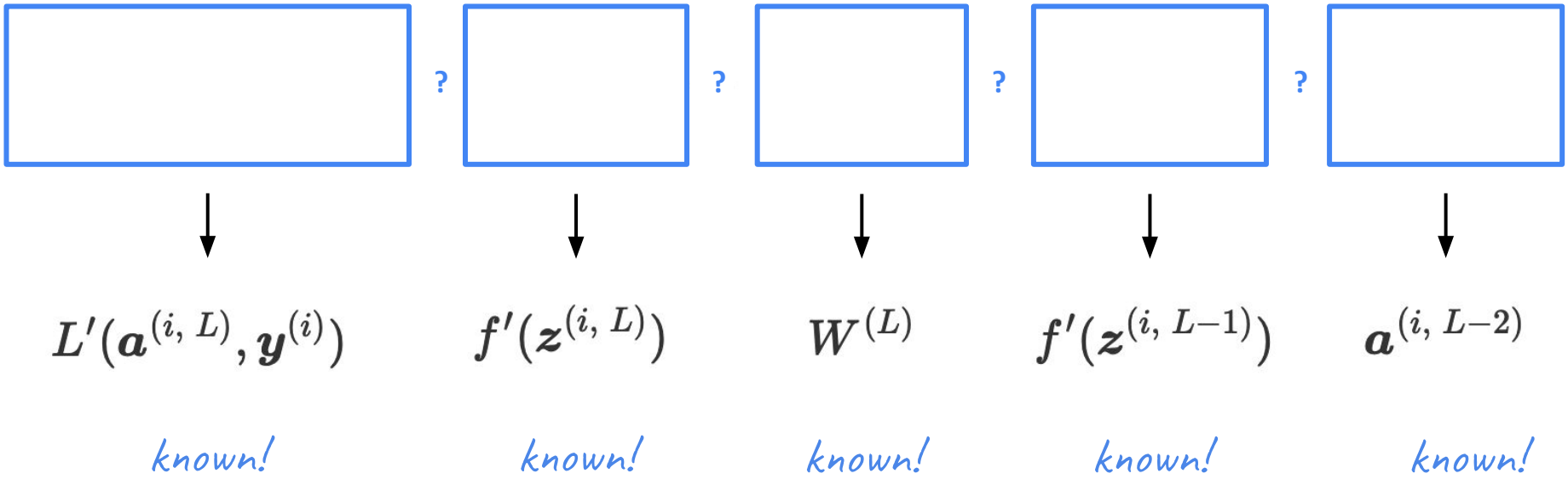
$$L'(\boldsymbol{a}^{(i,L)}, \boldsymbol{y}^{(i)}) \qquad f'(\boldsymbol{z}^{(i,L)}) \qquad \frac{\partial z_j^{(i,L)}}{\partial w_{kj}^{(L)}} = a_k^{(i,L-1)}$$

known! known! known!

Before Last Layer

From the cost's equation, applying the chain rule:

$$\frac{\partial \text{ Cost}}{\partial W^{(L-1)}} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial L(f(\boldsymbol{z}^{(i,\,L)}), \boldsymbol{y}^{(i)})}{\partial W^{(L-1)}}$$



Before-before last layer?

Before-before-last layer

$$\frac{\partial \text{ Cost}}{\partial W^{(L-2)}} = \frac{1}{m}\sum_{i=1}^{m} \frac{\partial L(f(\boldsymbol{z}^{(i,\,L)}), \boldsymbol{y}^{(i)})}{\partial W^{(L-2)}}$$

$$\frac{\partial L(f(\boldsymbol{z}^{(i,\,L)}), \boldsymbol{y}^{(i)})}{\partial f(\boldsymbol{z}^{(i,\,L)})} \cdot \frac{\partial f(\boldsymbol{z}^{(i,\,L)})}{\partial \boldsymbol{z}^{(i,\,L)}} \cdot \frac{\partial \boldsymbol{z}^{(i,\,L)}}{\partial \boldsymbol{a}^{(i,\,L-1)}} \cdot \frac{\partial \boldsymbol{a}^{(i,\,L-1)}}{\partial \boldsymbol{z}^{(i,\,L-1)}} \cdot \frac{\partial \boldsymbol{z}^{(i,\,L-1)}}{\partial \boldsymbol{a}^{(i,\,L-2)}} \cdot \frac{\partial \boldsymbol{a}^{(i,\,L-2)}}{\partial \boldsymbol{z}^{(i,\,L-2)}} \cdot \frac{\partial \boldsymbol{z}^{(i,\,L-2)}}{\partial W^{(L-2)}}$$

$$\downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow$$

$$L'(\boldsymbol{a}^{(i,\,L)}, \boldsymbol{y}^{(i)}) \qquad f'(\boldsymbol{z}^{(i,\,L)}) \qquad W^{(L)} \qquad f'(\boldsymbol{z}^{(i,\,L-1)}) \qquad W^{(L-1)} \qquad f'(\boldsymbol{z}^{(i,\,L-2)}) \qquad \boldsymbol{a}^{(i,\,L-3)}$$

We can see a pattern!

Memoization (and it's not a typo!)

Optimization technique to make computations faster, in particular by reusing previous calculations.

$$\frac{\partial \text{ Cost}}{\partial W^{(4)}} = \frac{1}{m} \sum L'(\boldsymbol{a}^{(4)}, \boldsymbol{y}) \cdot f'(\boldsymbol{z}^{(4)}) \cdot \boldsymbol{a}^{(3)}$$

$$\frac{\partial \text{ Cost}}{\partial W^{(3)}} = \frac{1}{m} \sum L'(\boldsymbol{a}^{(4)}, \boldsymbol{y}) \cdot f'(\boldsymbol{z}^{(4)}) \cdot W^{(4)} \cdot f'(\boldsymbol{z}^{(3)}) \cdot \boldsymbol{a}^{(2)}$$

$$\frac{\partial \text{ Cost}}{\partial W^{(2)}} = \frac{1}{m} \sum L'(\boldsymbol{a}^{(4)}, \boldsymbol{y}) \cdot f'(\boldsymbol{z}^{(4)}) \cdot W^{(4)} \cdot f'(\boldsymbol{z}^{(3)}) \cdot W^{(3)} \cdot f'(\boldsymbol{z}^{(2)}) \cdot \boldsymbol{a}^{(1)}$$

$$\frac{\partial \text{ Cost}}{\partial W^{(1)}} = \frac{1}{m} \sum L'(\boldsymbol{a}^{(4)}, \boldsymbol{y}) \cdot f'(\boldsymbol{z}^{(4)}) \cdot W^{(4)} \cdot f'(\boldsymbol{z}^{(3)}) \cdot W^{(3)} \cdot f'(\boldsymbol{z}^{(2)}) \cdot W^{(2)} \cdot f'(\boldsymbol{z}^{(1)}) \cdot \boldsymbol{x}$$

Let's rewrite using δ

$$\delta^{(4)} = f'(z^{(4)}) \odot L'(a^{(4)}, y)$$

$$\delta^{(3)} = f'(z^{(3)}) \odot [\, W^{(4)}\, \delta^{(4)} \,]$$

$$\delta^{(2)} = f'(z^{(2)}) \odot [\, W^{(3)}\, \delta^{(3)} \,]$$

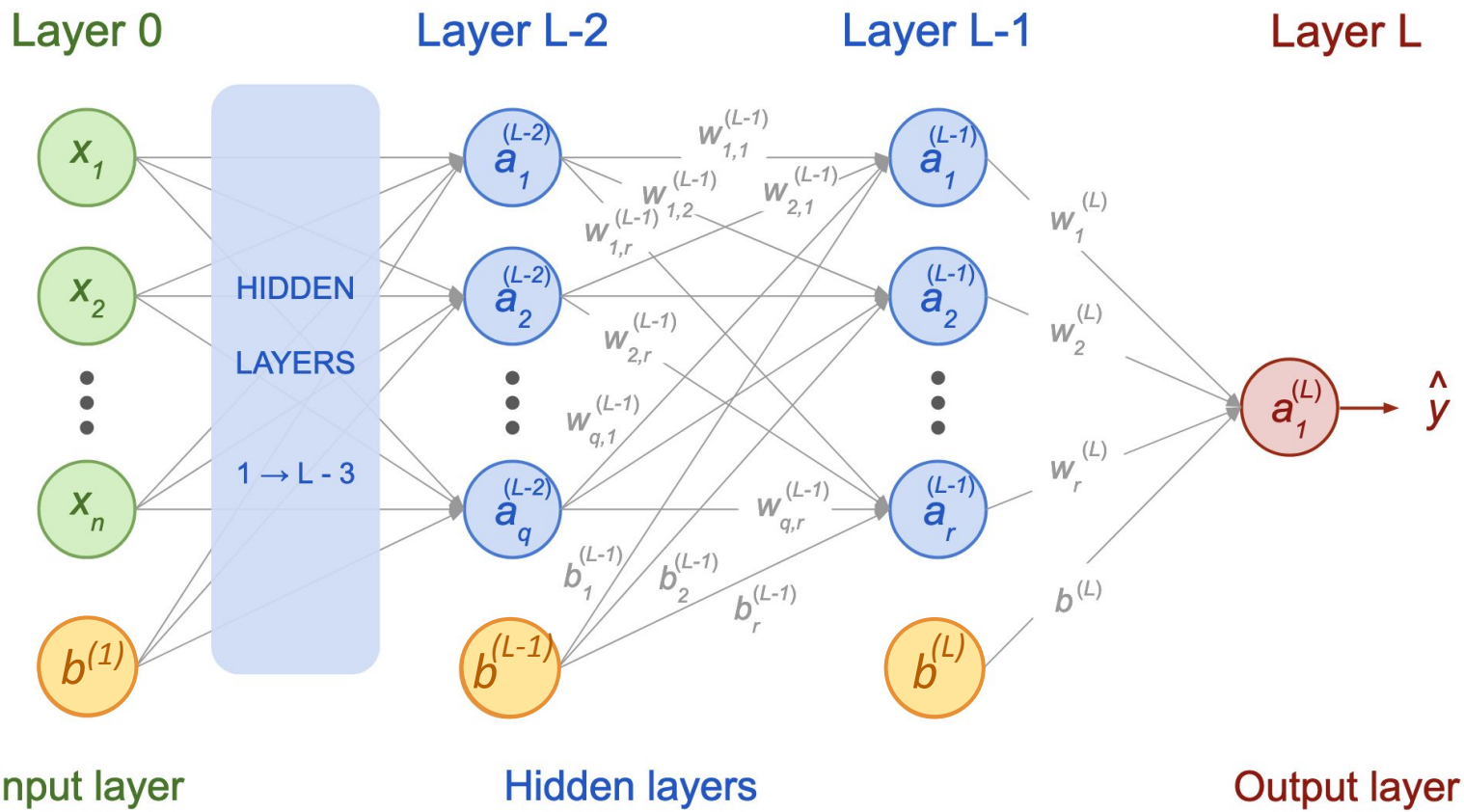
$$\frac{\partial \text{ Cost}}{\partial W^{(4)}} = \frac{1}{m} \sum L'(a^{(4)}, y) \cdot f'(z^{(4)}) \cdot a^{(3)}$$

$$\frac{\partial \text{ Cost}}{\partial W^{(3)}} = \frac{1}{m} \sum L'(a^{(4)}, y) \cdot f'(z^{(4)}) \cdot W^{(4)} \cdot f'(z^{(3)}) \cdot a^{(2)}$$

$$\frac{\partial \text{ Cost}}{\partial W^{(2)}} = \frac{1}{m} \sum L'(a^{(4)}, y) \cdot f'(z^{(4)}) \cdot W^{(4)} \cdot f'(z^{(3)}) \cdot W^{(3)} \cdot f'(z^{(2)}) \cdot a^{(1)}$$

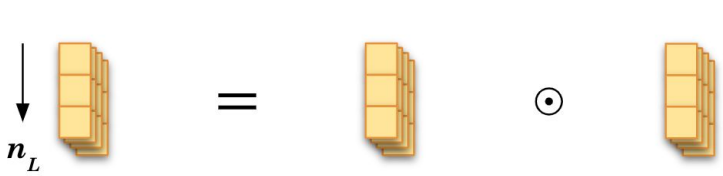
$$\frac{\partial \text{ Cost}}{\partial W^{(1)}} = \frac{1}{m} \sum L'(a^{(4)}, y) \cdot f'(z^{(4)}) \cdot W^{(4)} \cdot f'(z^{(3)}) \cdot W^{(3)} \cdot f'(z^{(2)}) \cdot W^{(2)} \cdot f'(z^{(1)}) \cdot x$$

What would be the dimensions of the δ errors?



Last layer: δ error

Column vector of n_L elements:



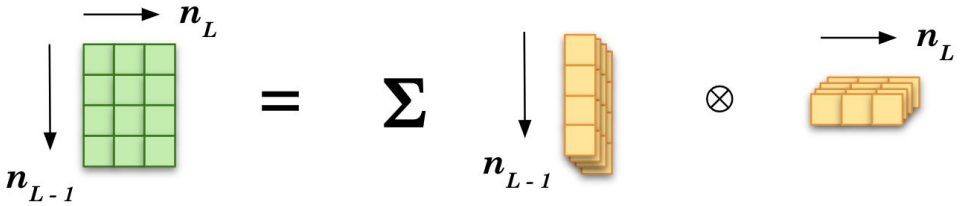
$$\boldsymbol{\delta}^{(i,\,L)} = f'(\boldsymbol{z}^{(i,\,L)}) \odot L'(\boldsymbol{a}^{(i,\,L)}, \boldsymbol{y}^{(i)})$$

Element-wise multiplication

Depends on the sample index i (sheets)

Cost derivatives

Same dimension as the matrix



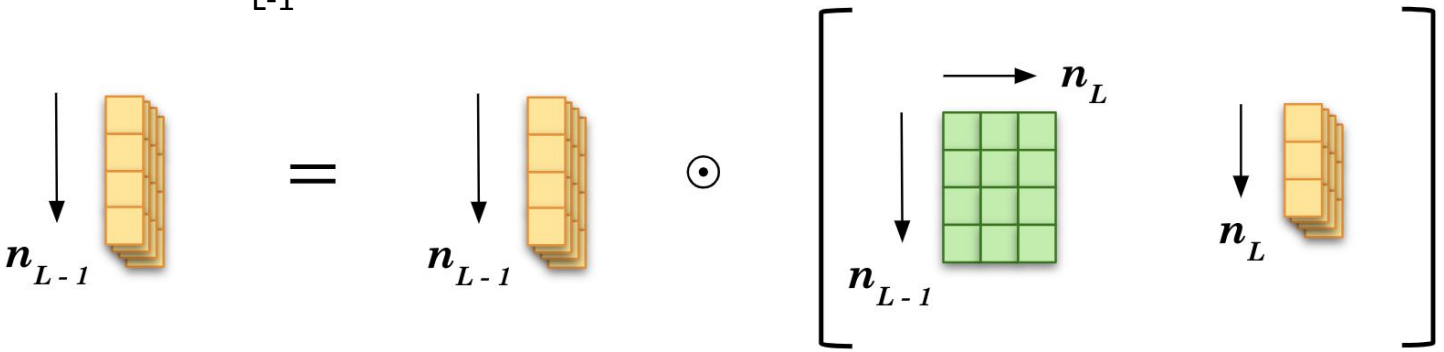
$$\frac{\partial\,\text{Cost}}{\partial\,W^{(L)}} = \frac{1}{m}\sum_{i=1}^{m} \boldsymbol{a}^{(i,\,L-1)} \otimes \left[\,\boldsymbol{\delta}^{(i,\,L)}\,\right]$$

Outer product

No dependence on sample index i

Before-last layer: δ error

Error is of dimension n_{L-1} :

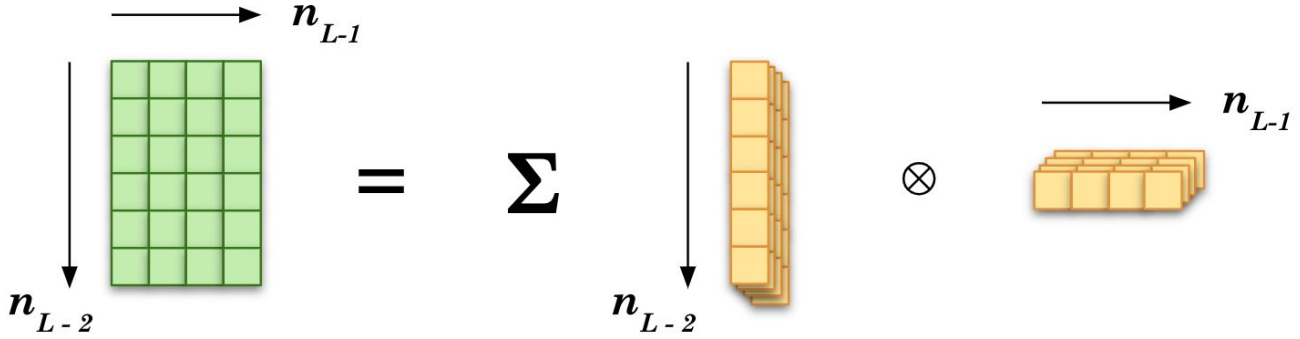


$$\boldsymbol{\delta}^{(i,\,L-1)} = f'(\boldsymbol{z}^{(i,\,L-1)}) \odot \left[W^{(L)}\, \boldsymbol{\delta}^{(i,\,L)} \right]$$

matrix multiplication

Before-last layer: cost derivatives

Matrix of dimension $n_{L-2} \times n_{L-1}$



$$\frac{\partial \text{ Cost}}{\partial W^{(L-1)}} = \frac{1}{m} \sum_{i=1}^{m} \boldsymbol{a}^{(i,\, L-2)} \otimes \left[\boldsymbol{\delta}^{(i,\, L-1)} \right]$$

outer product

Let’s wrap up

Neural Network Training

Step 1: Forward Propagation: list of predictions: $f(\boldsymbol{z}^{(i,\,L)}) = \boldsymbol{a}^{(i,\,L)} = \hat{\boldsymbol{y}}^{(i)}$

Step 2: Back Propagation: chain rule on the $\text{Cost} = \dfrac{1}{m}\displaystyle\sum_{i=1}^{m} L\Big(\, f\Big(\, \boldsymbol{z}^{(i,\,L)}\Big), \boldsymbol{y}^{(i)}\Big)$

⇒ all errors: $\boldsymbol{\delta}^{(i,\,L)} = f'(\boldsymbol{z}^{(i,\,L)}) \odot L'(\boldsymbol{a}^{(i,\,L)}, \boldsymbol{y}^{(i)})$

$$\boldsymbol{\delta}^{(i,\,\ell)} = f'(\boldsymbol{z}^{(i,\,\ell)}) \odot \Big[\, W^{(\ell+1)}\, \boldsymbol{\delta}^{(i,\,\ell+1)} \,\Big]$$

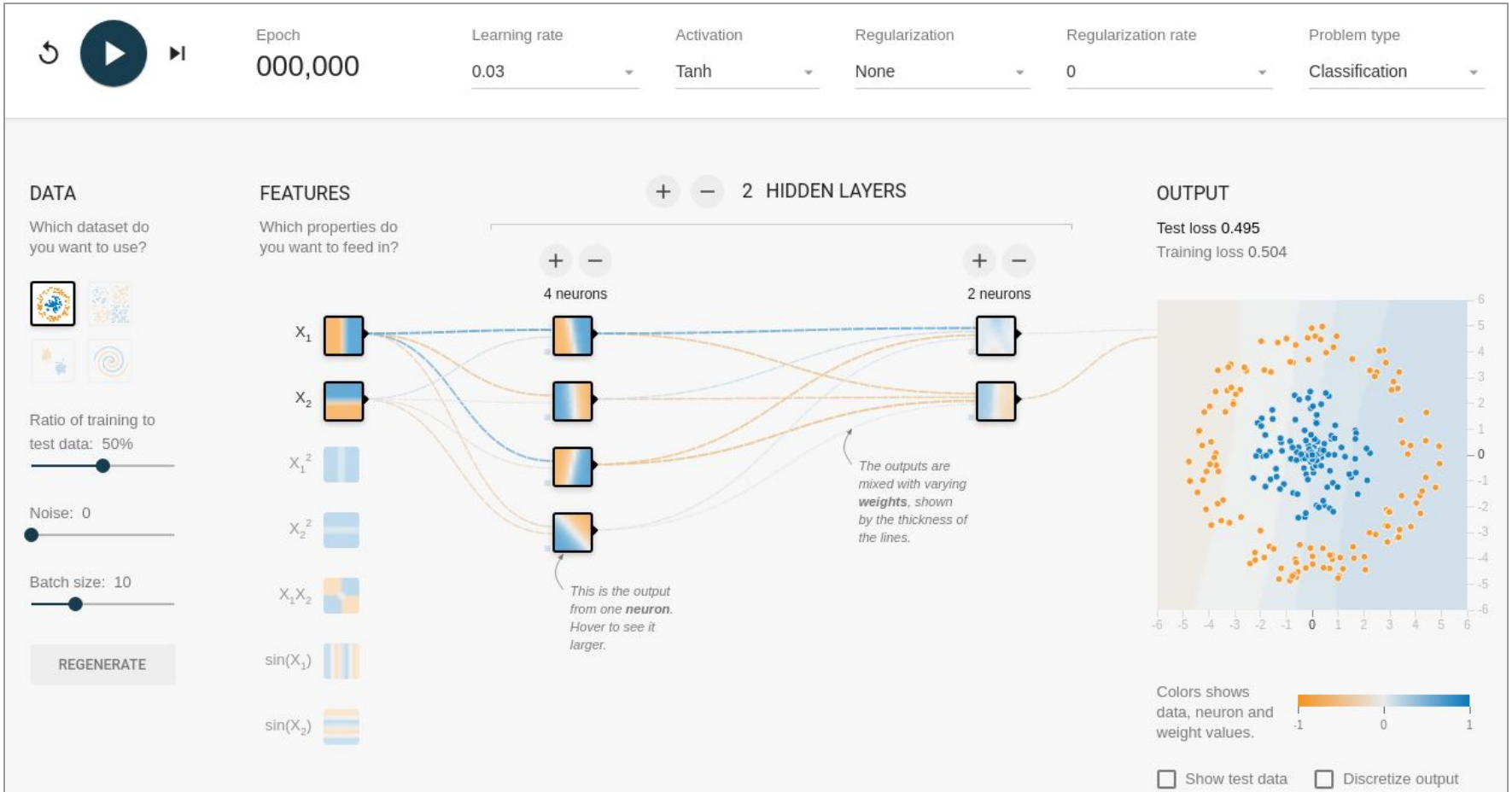
⇒ derivatives: $\dfrac{\partial\, \text{Cost}}{\partial\, W^{(\ell)}} = \dfrac{1}{m}\displaystyle\sum_{i=1}^{m} \boldsymbol{a}^{(i,\,\ell-1)} \otimes \Big[\, \boldsymbol{\delta}^{(i,\,\ell)} \,\Big]$ $\qquad$ $\dfrac{\partial\, \text{Cost}}{\partial\, \boldsymbol{b}^{(\ell)}} = \dfrac{1}{m}\displaystyle\sum_{i=1}^{m} \boldsymbol{\delta}^{(i,\,\ell)}$

Step 3: Gradient Descent: $W^{(\ell)} \leftarrow W^{(\ell)} - \alpha \dfrac{\partial\, \text{Cost}}{\partial W^{(\ell)}}$ $\qquad$ $\boldsymbol{b}^{(\ell)} \leftarrow \boldsymbol{b}^{(\ell)} - \alpha \dfrac{\partial\, \text{Cost}}{\partial \boldsymbol{b}^{(\ell)}}$

Repeat 1 → 3 $\qquad$ **Exit** after $N^{\text{iterations}}$ or if **all** derivatives → 0

TensorFlow Playground

playground.tensorflow.org



You now know the maths behind neural networks!

Extra

What is intelligence?

Definition(s) of intelligence (continued)

Etymology

from Latin *intelligere* → "to understand, comprehend, come to know"

A synthesis attempt

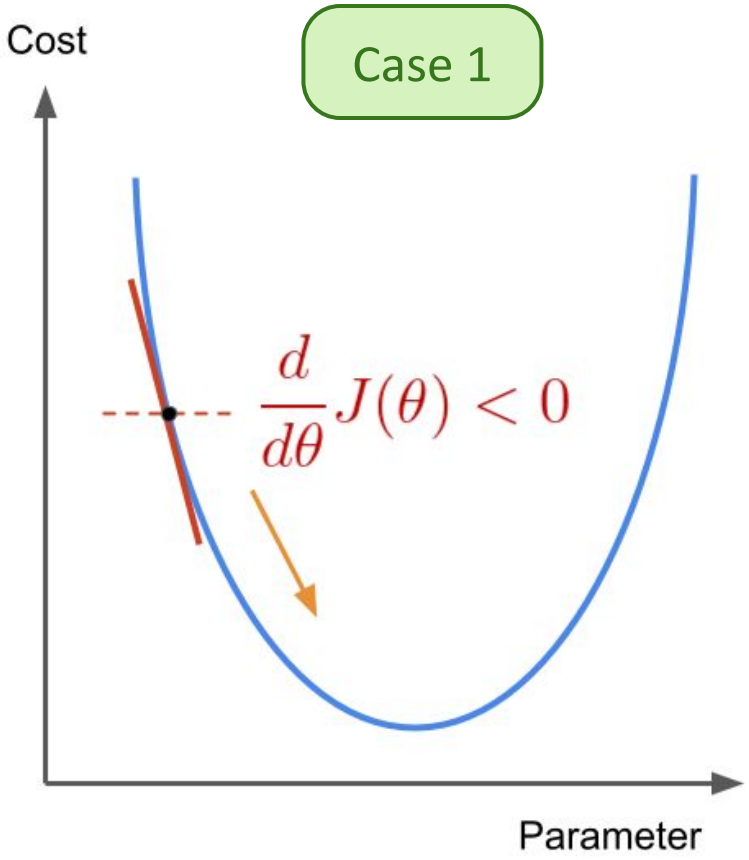
Intelligence measures an agent's ability to achieve goals
in a wide range of environments.

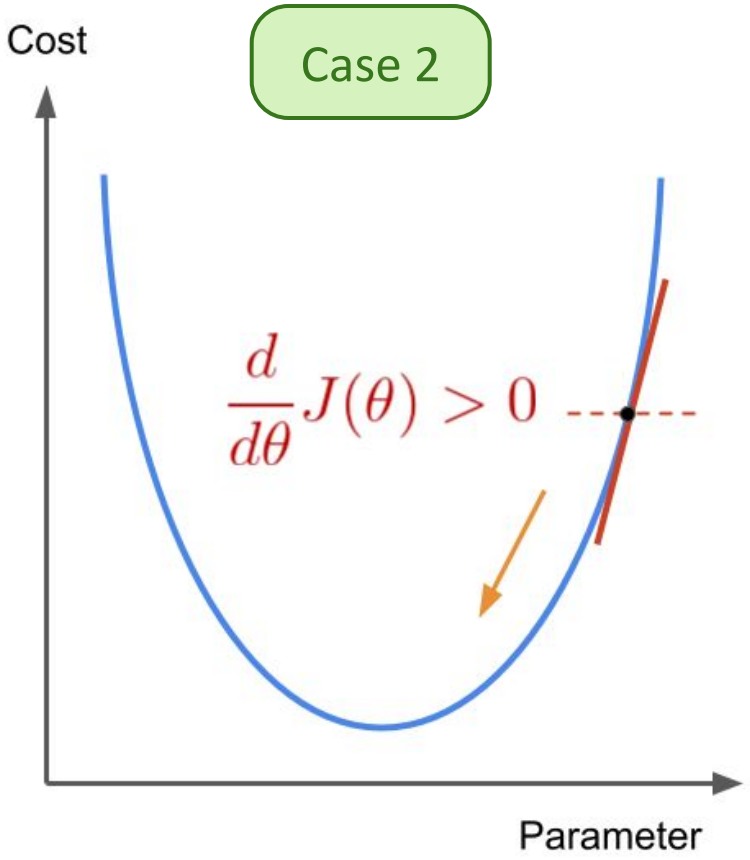
Shane Legg, Marcus Hutter arXiv:0712.3329

"A fundamental problem in artificial intelligence is that
nobody really knows what intelligence is."

Why a minus sign?

$$\theta_1' = \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$





Logistic Regression

Definition

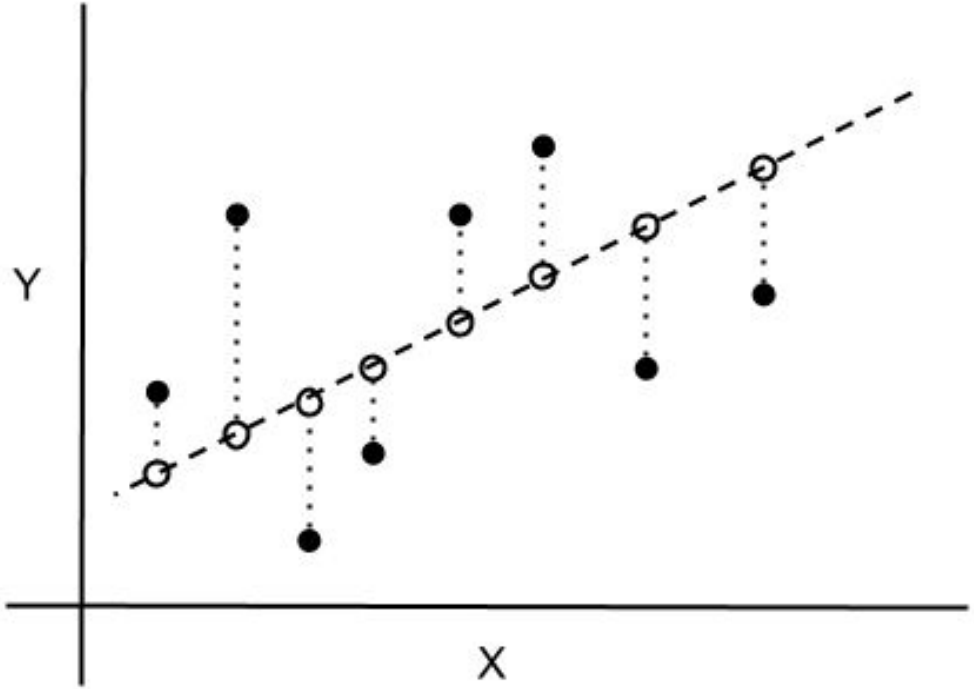
Statistical model used in machine learning to make a prediction from one or more independent variables about a categorical variable.

variable taking limited number of possible values:
true or false, blue/red/green, 1 or 0.

Binary classification ⇒ two categorical variables

Multi-class classification ⇒ more than two categorical variables

Chi-square statistics



● Observed Values

○ Predicted Values

- - - - Regression Line

····· Y Scale Difference Between Observed and Predicted Values

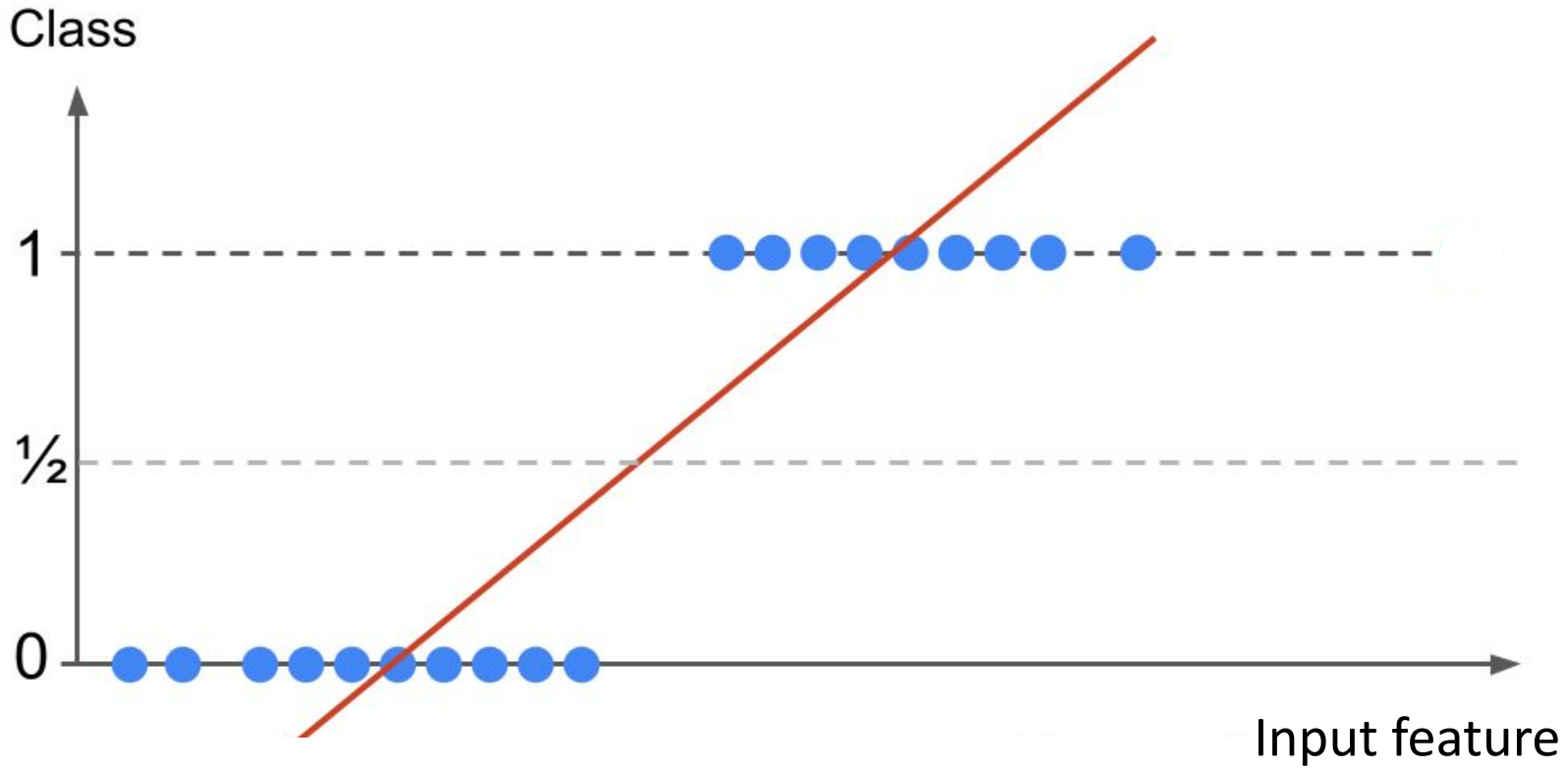
$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

χ^2 = chi-squared

O_i = observed values

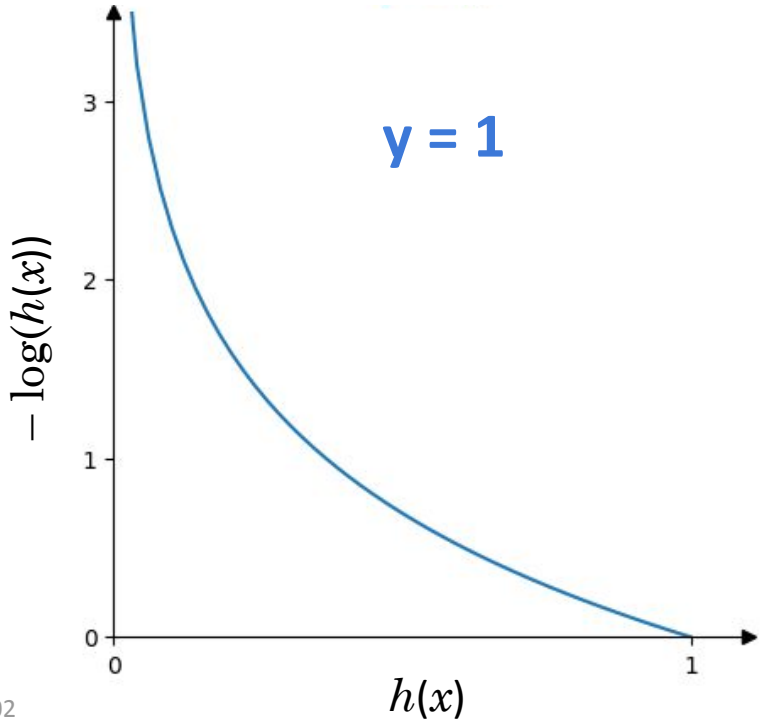
E_i = expected values

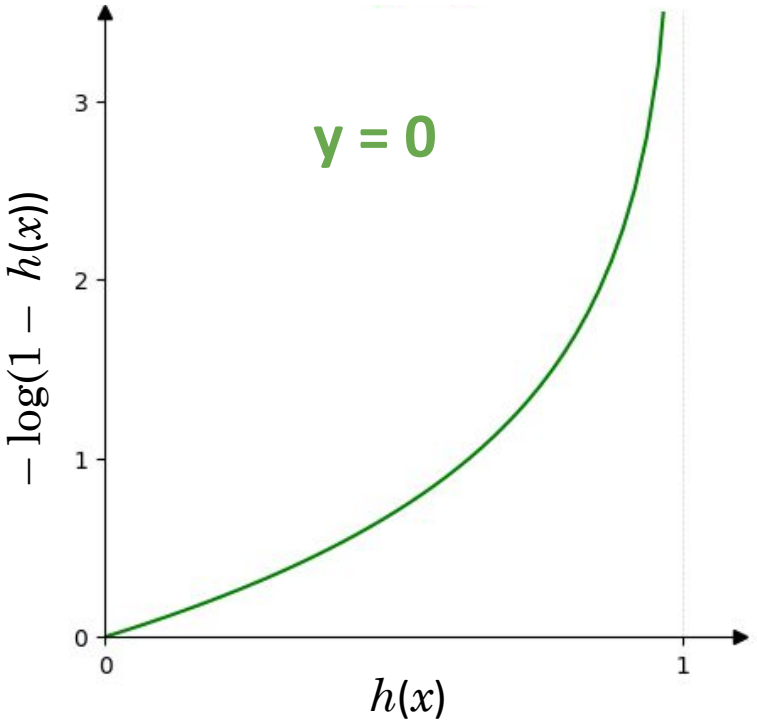
Linear Regression tools?



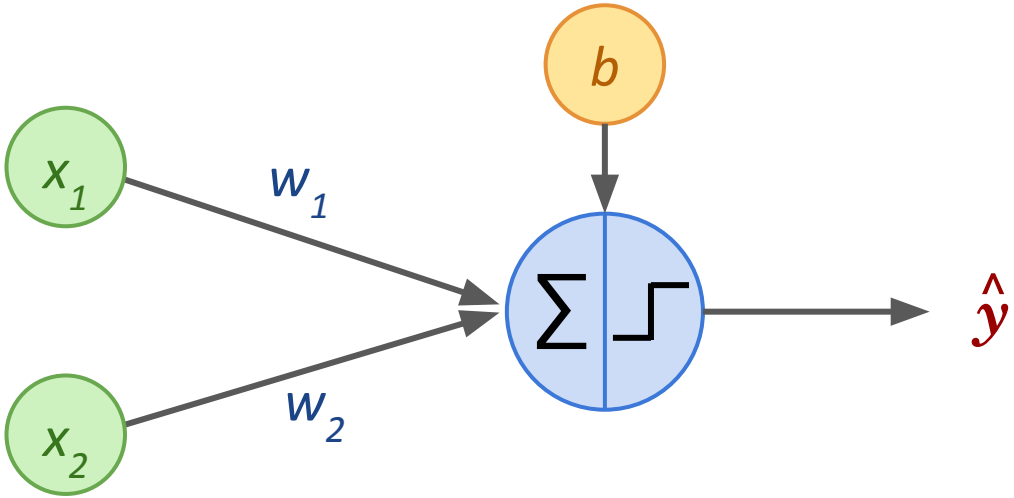
Cost Function for Logistic Regression

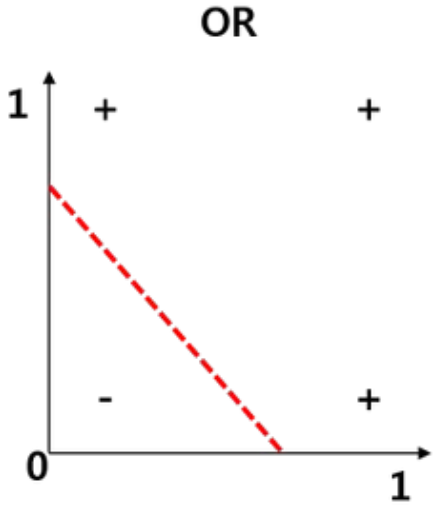
$$\text{Cost} = -\frac{1}{m}\sum_{i=1}^{m}\left[\; y^{(i)}\log(\;h(x^{(i)})) \;+\; (1-y^{(i)})\log(1-\;h(x^{(i)}))\;\right]$$





Perceptron to solve “OR”



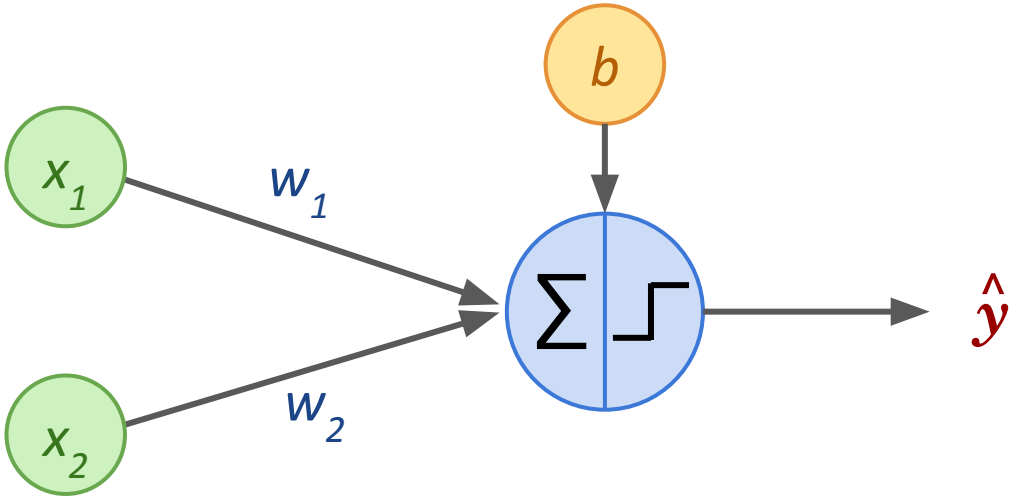


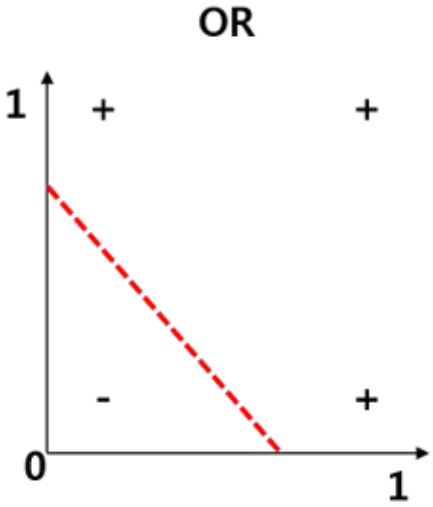
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Exercise:

What would be the weights w_1 , w_2 and bias b to encode the **OR** function? (using the Heaviside step function)

Perceptron to solve “OR”





Answer:

$$b = -0.5$$
$$w_1 = 1$$
$$w_2 = 1$$

one possibility!

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

More on Activation Functions

Sigmoid

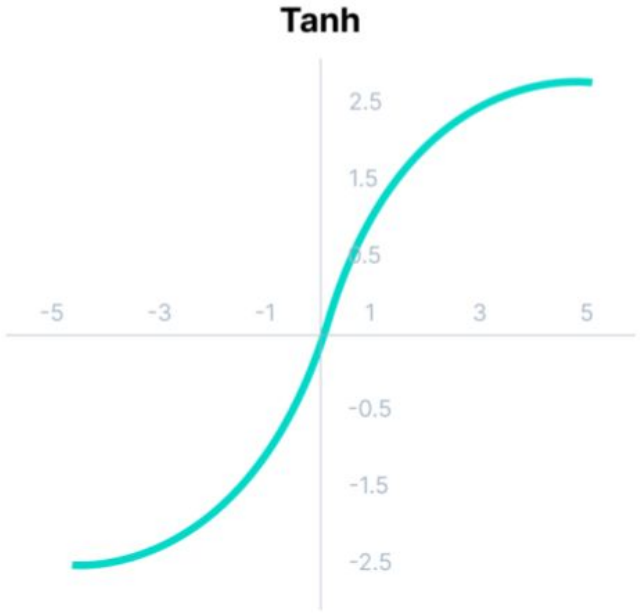
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

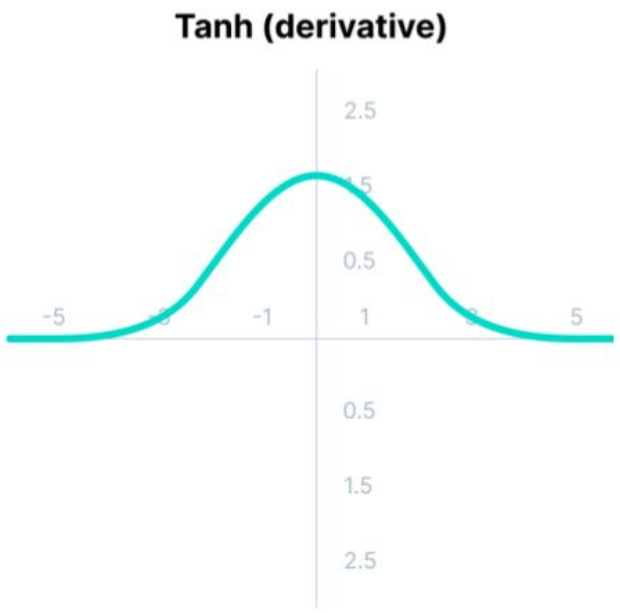
Pros: differentiable + bounded → generate probability-equivalent as output

Cons: derivative vanishes at ± ∞ → *Vanishing Gradient* problem

Hyperbolic Tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



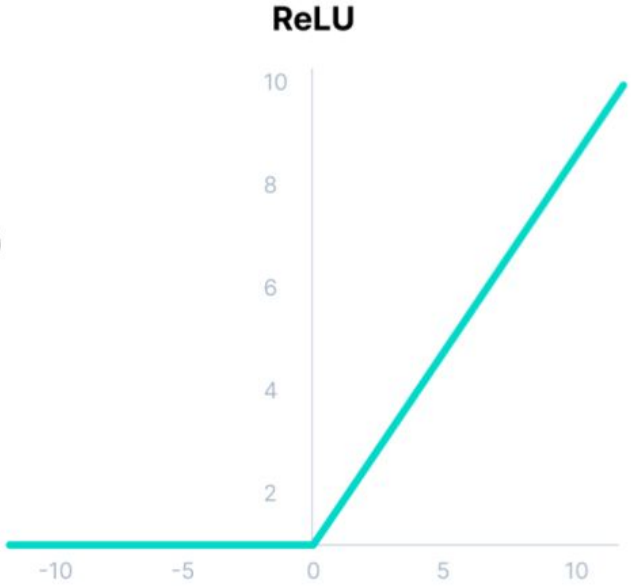


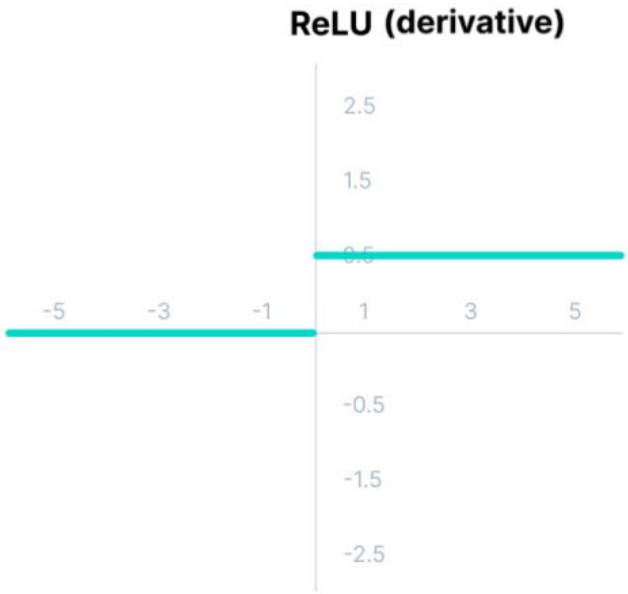
Pros: zero-centered ⇒ mapping more straightforward
mean of the output values is close to zero ⇒ easier learning

Cons: steeper (risk of jumps during Descent) + also *Vanishing Gradient* problem

Rectified Linear Unit (ReLU)

$$f(z) = \max(0, z)$$





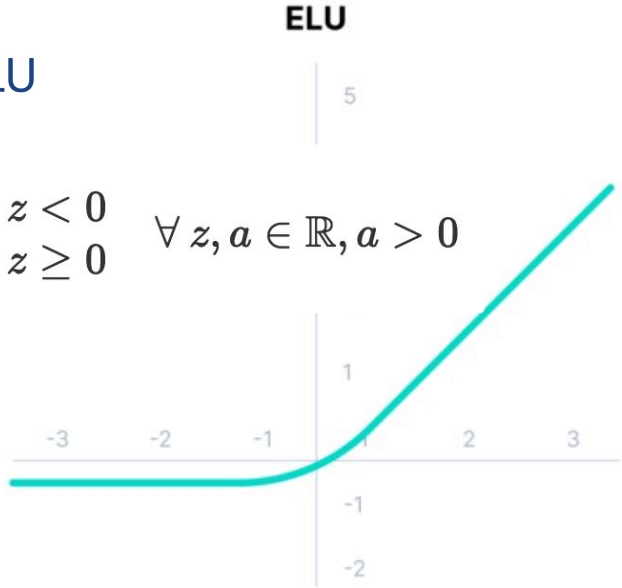
Pros: huge gain in computational efficiency

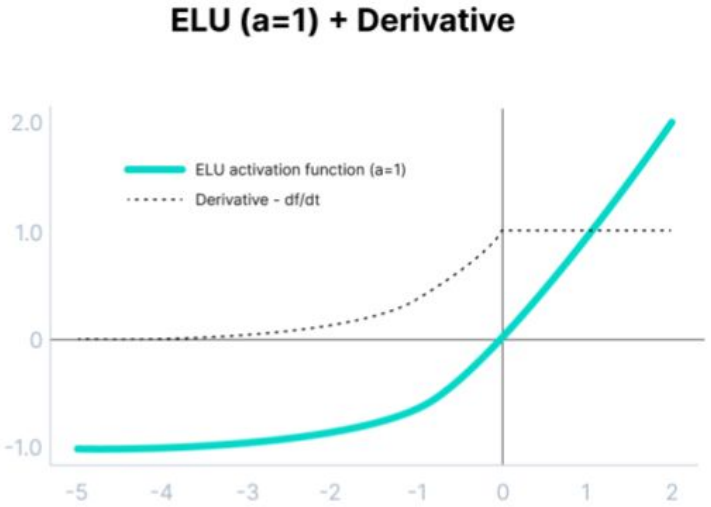
50% of activation units active (*sparse activation*) ⇒ quick computation

Cons: not zero-centered, not bounded, not diff.[able] at zero '*dying ReLU problem*'

Exponential Linear Units (ELUs) variant of ReLU

$$\mathrm{ELU}(z) = \begin{cases} a(e^z - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases} \qquad \forall\, z, a \in \mathbb{R}, a > 0$$



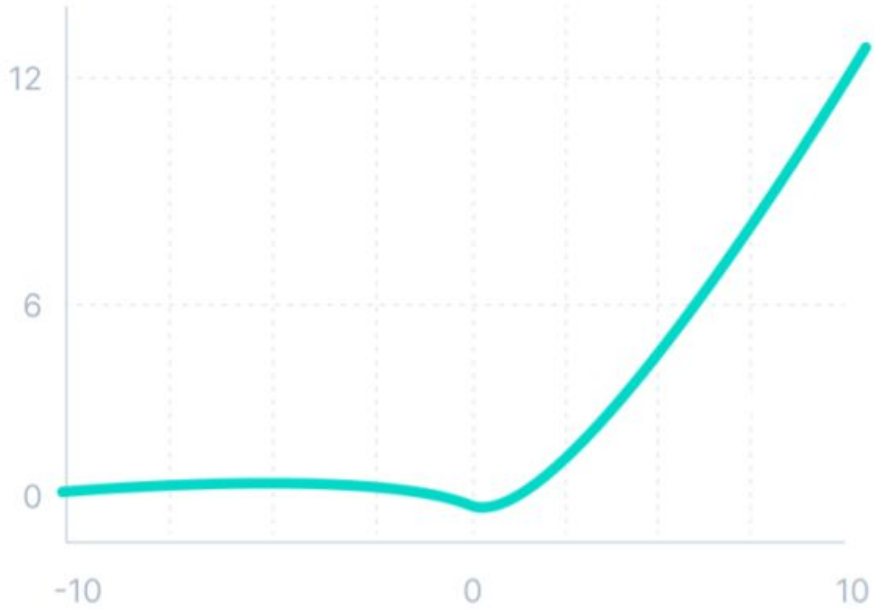


Pros: no 'kick' but smooth decrease
shown to converge faster and give more accurate results

Cons: parameter α needs to be tuned + *Exploding Gradient* problem

Sigmoid Linear Unit (SiLU) and Swish

$$f(z) = \frac{z}{1 + e^{-z}}$$



Pros: fully differentiable, smooth & non monotonic
small negative values not zeroed → better modelling of the data

~~**Cons**~~ → **Warning:** worth it for NN with more than 40 layers